Reproduction Information

Correlated uncertainty propagation enables multi-impact decision support for electrical system decarbonization

Amir M. Gazar[1,2], Chloe Jackson[3], Georgia Mavrommati[3], Rich B. Howarth[4], and Ryan S.D. Calder[1,2,5,*]

[1]Department of Population Health Sciences, Virginia Tech, Blacksburg, VA, 24061, USA
[2]Global Change Center, Virginia Tech, Blacksburg, VA, 24061, USA
[3]School for the Environment, University of Massachusetts Boston, Boston, MA 02125, USA
[4]Environmental Program, Dartmouth College, Hanover, NH, 03755, USA
[5]Department of Civil and Environmental Engineering, Duke University, Durham, NC, 27708, USA
[*]Corresponding author: rsdc@vt.edu

Contents

1 Conceptual Overview and Code Availability

1.1 Code availability

The code for this study is available on the GitHub repository at https://github.com/amirgazar/Decarbonization-Tradeoffs. The repository is organized into four top-level folders: `1 Decarbonization Pathways` (pathway definitions and hourly capacity expansion), `2 Generation Expansion Model` (the PHASED generation model, including wind/solar/SMR/fossil-fuel modules, imports, storage, randomization, and the dispatch curve), `3 Total Costs` (per-component cost scripts and the consolidated total-cost and uncertainty analysis), and `4 External Data` (referenced datasets such as NREL ATB, ISO-NE, U.S. EIA, U.S. EPA, and ORNL LandScan). Python notebooks for figure generation, post-processing, and non-monetized ecological impact estimation accompany the R pipeline.

1.2 Conceptual Overview

The framework integrates key components across three stages: (1) Input Variables, which include the eight decarbonization pathways (A, B1, B2, B3, C1, C2, C3, D; e.g., new offshore wind, transmission lines to/from Quebec, and new small modular reactors) and external data sources (ISO-NE demand, EIA and EPA datasets, IEA resources, and NREL ATB); (2) the PHASED generation model (Probabilistic Hourly Assessment of System Energy and Decarbonization), which computes hourly demand, clean and fossil fuel generation, imports, energy storage, curtailment, and unmet demand at the Regional Transmission Organization (RTO) level under Monte Carlo uncertainty; and (3) Total Costs, which are determined through cost modules covering CAPEX, FOM, VOM, fuel, imports, GHG emissions, air pollutant emissions, unmet demand penalties, and beyond-the-fence-line (Canadian hydropower) costs, to estimate net present value (NPV) of total social costs under three discount rates (1.5%, 2%, and 2.5%).

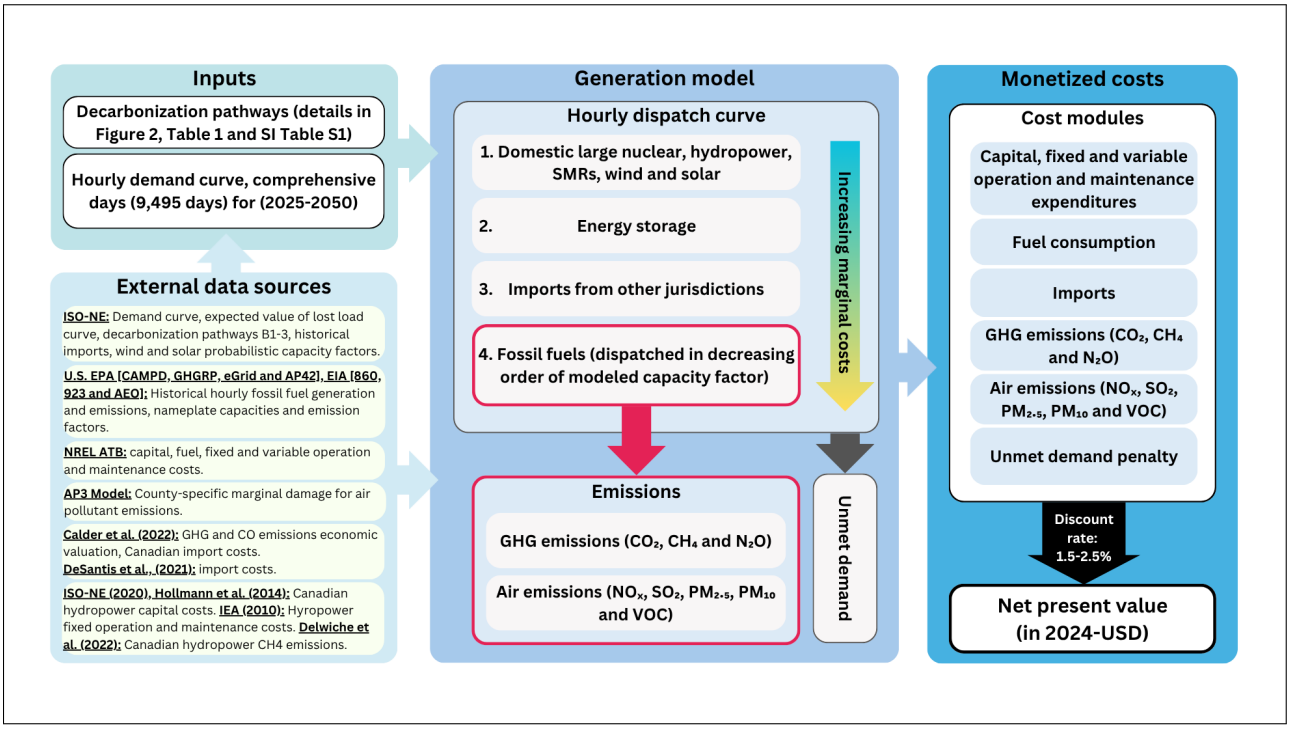

Figure 1: Conceptual overview of the modeling framework.

2 Configuration and Setup

This document presents a comprehensive guide outlining the sequential process required to execute and replicate the PHASED generation model, the total cost calculation, and the non-monetized ecological impact estimates presented in our article. Additionally, we provide a conceptual overview of code functionality in Section 1.2.

2.1 Hardware and Operating System

The code was developed on a MacBook Pro with an Apple M1 Pro chip (8-core CPU, 16 GB of memory) running macOS 14.5 `Sonoma`. To run the PHASED dispatch curve at scale, we used Virginia Tech's Advanced Research Computing (ARC) resources. The following SLURM specifications were used for each simulation batch:

```
#SBATCH --partition=normal_q
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=50
#SBATCH --cpus-per-task=1
#SBATCH --time=24:00:00
```

2.2 Software Versions and Dependencies

We used R version 4.4.2 `(2024-10-31)` for the x86_64-apple-darwin20 platform, executed within RStudio version 2024.09.1 Build 394. Python notebooks (figure generation, ecological impacts, and uncertainty analysis) were run under Python 3.11 with Jupyter. Table 1 lists the R packages and versions used by the model and cost pipeline.

Table 1: R packages and versions used by the PHASED pipeline.

Package	Version
data.table	1.14.8
lubridate	1.9.3
dplyr	1.1.2
tidyr	1.3.0
readxl	1.4.3
readr	2.1.4
purrr	1.0.2
ggplot2	3.4.2
scales	1.2.1
zoo	1.8.12
gridExtra	2.3
sf	1.0.14
tigris	2.1
fredr	2.1.0
randtoolbox	2.0.4
httr	1.4.6
jsonlite	1.8.7
htmltools	0.5.5
future, future.apply	1.33.0, 1.11.0

Key Python packages used by the notebooks include `pandas`, `numpy`, `matplotlib`, `seaborn`, `scipy`, `scikit-learn` (for the random-forest sensitivity analysis), `geopandas`, `shapely`, and `rasterio` (for the viewshed and emissions county-level maps).

2.3 Install Time

Installation of R, RStudio, and the listed packages typically takes a few minutes on a standard workstation. Python package installation is similarly brief; `rasterio` and `geopandas` are the slowest because they depend on GDAL system libraries.

2.4 Run Time

Approximate run times observed on Virginia Tech's ARC and on a MacBook Pro M1:

- Full PHASED dispatch curve (9,495 comprehensive days, all eight pathways): ≈15 minutes per simulation per pathway on ARC. The full Monte Carlo ensemble (thousands of simulations) is run as an embarrassingly parallel batch across nodes.

- Per-component cost scripts (`1_CAPEX_FOM_ATB_Fossil.R` through `12_Other_S3_CAN_Hydro.R`): a few minutes each on a laptop.
- Total-cost consolidation and uncertainty analysis (`13_Total_Costs_and_Uncertainty_Analysis.R` and `Uncertainty_analysis.ipynb`): under 10 minutes on a laptop.
- Figure-generation Python notebooks: under a minute each, except `Visual_impacts.ipynb` (viewshed exposure), which is dominated by raster masking and takes several minutes.

Depending on dataset size and hardware configuration, these times may vary.

3 Model Formulation

This section provides the mathematical formulation of the **PHASED** (Probabilistic Hourly Assessment of System Energy and Decarbonization) generation model. The notation here mirrors that used in the Supplemental Information of the manuscript and the corresponding R implementation (`dispatch_curve_base_v3.R`).

3.1 Notation

3.1.1 Sets and Indices

$t \in \mathcal{T}$	hours,	$u \in \mathcal{U}$	fossil generating units,
$g \in \mathcal{G}$	generation technologies,	$j \in \mathcal{J}$	interties,
$\omega \in \Omega$	Monte Carlo draw,	$y \in \mathcal{Y}$	years,
$p \in \mathcal{P}$	decarbonization pathways.		

$$\mathcal{G} = \{\text{Solar}, \text{Wind}_{\text{on}}, \text{Wind}_{\text{off}}, \text{Nuclear}, \text{Hydro}, \text{Biomass}, \text{SMR}, \text{Fossil}\},$$
$$\mathcal{J} = \{\text{HQ}, \text{NYISO}, \text{NBSO}\},$$
$$\mathcal{Y} = \{2025, \ldots, 2050\},$$
$$\mathcal{P} = \{\text{A}, \text{B1}, \text{B2}, \text{B3}, \text{C1}, \text{C2}, \text{C3}, \text{D}\}.$$

3.1.2 Parameters (selected)

D_t	demand (MWh),
$\text{Cap}_{g,t}$	installed capacity (MW),
$\text{CF}_{g,t}(\omega) \in [0,1]$	availability factor,
$\text{Cap}^{\text{LT}}_{j,t}$, $\text{Cap}^{\text{spot}}_{j,t}$	import capacities (MW),
$\overline{I}_t$	imports ceiling (MWh),
$\eta \in (0,1]$	one-way efficiency,
$\rho \in (0,1]$	hourly retention factor,
$E_{\max,t}$	storage energy cap (MWh),
$P_{\max,t}$	storage inverter/power cap (MW),
$\underline{P}_u$, $\overline{P}_u$	unit min/max (MW),
$\overline{R}_u$	ramp limit (MWh per hour),
$\chi_{u,t} \in \{0,1\}$	availability/retirement,
κ_j	import price (USD/MWh),
f_u, v_g	fuel and VOM rates (USD/MWh),
$s^{\text{GHG}}_{k,y}$	social cost for $k \in \{\text{CO}_2, \text{CH}_4, \text{N}_2\text{O}\}$,
v^k_t	damage factor for pollutant k,
π_{short}	value of lost load (USD/MWh),
r, y_0	discount rate, base year.

3.1.3 State and Decision Variables

$$G_{g,t} \geq 0 \quad \text{non-fossil generation (MWh)}, \qquad G_{u,t} \geq 0 \quad \text{fossil unit generation (MWh)},$$
$$I^{\mathrm{LT}}_{j,t},\, I^{\mathrm{spot}}_{j,t} \geq 0 \quad \text{imports (MWh)}, \qquad \mathrm{SOC}_t \in [0, E_{\max,t}] \quad \text{state of charge (MWh)},$$
$$c_t, d_t \geq 0 \quad \text{charge/discharge (MWh)}, \qquad \mathrm{Curt}_t, \mathrm{Short}_t \geq 0 \quad \text{curtailment, shortage (MWh)}.$$

3.2 Stochastic availability sampling (percentile mapping)

Let the random percentile sequence for simulation index ω be $p_\omega \in \{1, \ldots, 99\}^{250}$ (see Section 5.8). For each hour t,

$$\mathrm{CF}_{\mathrm{Solar},t}(\omega) = \mathrm{CF}^{\mathrm{tbl}}_{\mathrm{Solar}}(\mathrm{DayLabel}(t),\, \mathrm{Hour}(t),\, p_\omega), \tag{1}$$
$$\mathrm{CF}_{\mathrm{Wind_{on}},t}(\omega) = \mathrm{CF}^{\mathrm{tbl}}_{\mathrm{Wind_{on}}}(\mathrm{DayLabel}(t),\, \mathrm{Hour}(t),\, p_\omega), \tag{2}$$
$$\mathrm{CF}_{\mathrm{Wind_{off}},t}(\omega) = \mathrm{CF}^{\mathrm{tbl}}_{\mathrm{Wind_{off}}}(\mathrm{DayLabel}(t),\, \mathrm{Hour}(t),\, p_\omega). \tag{3}$$

Imports percentiles are analogously sampled per intertie $j \in \{\mathrm{HQ}, \mathrm{NYISO}, \mathrm{NBSO}\}$ from the imports CF table:

$$\mathrm{CF}^{\mathrm{spot}}_{j,t}(\omega) = \mathrm{CF}^{\mathrm{tbl}}_{j}(\mathrm{DayLabel}(t),\, p_\omega). \tag{4}$$

Constant baseload availability factors apply for the remaining low-carbon technologies:
$\mathrm{CF}_{\mathrm{Nuclear}}, \mathrm{CF}_{\mathrm{Hydro}}, \mathrm{CF}_{\mathrm{Biomass}}, \mathrm{CF}_{\mathrm{SMR}}$ (with $\mathrm{CF}_{\mathrm{SMR}} = 0.90$).

3.3 Generation and Imports

3.3.1 Clean generation (hourly realized output)

$$G_{\mathrm{Solar},t} = \mathrm{Cap}_{\mathrm{Solar},t}\, \mathrm{CF}_{\mathrm{Solar},t}(\omega), \tag{5}$$
$$G_{\mathrm{Wind_{on}},t} = \mathrm{Cap}_{\mathrm{Wind_{on}},t}\, \mathrm{CF}_{\mathrm{Wind_{on}},t}(\omega), \tag{6}$$
$$G_{\mathrm{Wind_{off}},t} = \mathrm{Cap}_{\mathrm{Wind_{off}},t}\, \mathrm{CF}_{\mathrm{Wind_{off}},t}(\omega), \tag{7}$$
$$G_{\mathrm{Nuclear},t} = \mathrm{Cap}_{\mathrm{Nuclear},t}\, \mathrm{CF}_{\mathrm{Nuclear}}, \tag{8}$$
$$G_{\mathrm{Hydro},t} = \mathrm{Cap}_{\mathrm{Hydro},t}\, \mathrm{CF}_{\mathrm{Hydro}}, \tag{9}$$
$$G_{\mathrm{Biomass},t} = \mathrm{Cap}_{\mathrm{Biomass},t}\, \mathrm{CF}_{\mathrm{Biomass}}, \tag{10}$$
$$G_{\mathrm{SMR},t} = \mathrm{Cap}_{\mathrm{SMR},t}\, \mathrm{CF}_{\mathrm{SMR}}. \tag{11}$$

Define total clean output $G^{\mathrm{clean}}_t \equiv \sum_{g \in \mathcal{G} \setminus \{\mathrm{Fossil}\}} G_{g,t}$.

3.3.2 Imports (spot + long-term) and ceiling

Let $\overline{\mathrm{CF}}^{\mathrm{imp}} = 0.95$ be the maximum import CF (NREL ATB assumption). Then:

$$I^{\mathrm{LT}}_{j,t} = \mathrm{Cap}^{\mathrm{LT}}_{j,t}\, \overline{\mathrm{CF}}^{\mathrm{imp}}, \tag{12}$$
$$0 \leq I^{\mathrm{spot}}_{j,t} \leq \mathrm{Cap}^{\mathrm{spot}}_{j,t}\, \mathrm{CF}^{\mathrm{spot}}_{j,t}(\omega), \tag{13}$$
$$I_t = \sum_j \left(I^{\mathrm{LT}}_{j,t} + I^{\mathrm{spot}}_{j,t}\right), \qquad I_t \leq \overline{I}_t = \overline{\mathrm{CF}}^{\mathrm{imp}} \sum_j \left(\mathrm{Cap}^{\mathrm{LT}}_{j,t} + \mathrm{Cap}^{\mathrm{spot}}_{j,t}\right). \tag{14}$$

For NYISO and NBSO, the model treats spot CF as zero, so $I^{\mathrm{spot}}_{\mathrm{NYISO},t} = I^{\mathrm{spot}}_{\mathrm{NBSO},t} = 0$ and all flows enter through the long-term channel.

3.4 Battery storage (symmetric RTE, inverter/duration, retention)

3.4.1 Power cap from inverter and duration

Let the nameplate energy capacity be $E_{\max,t} = \mathrm{Cap}^{\mathrm{stor}}_t$ (MWh) and duration $H^{\mathrm{dur}} = 8\,\mathrm{h}$ (default in the implementation). With an optional inverter cap Inv_t from the `Inverter_MW` column of `Hourly_Installed_Capacity`, the derived power limit is

$$P_{\max,t} = \max\Big\{0,\ \min\Big\{\tfrac{E_{\max,t}}{H^{\mathrm{dur}}},\ \mathrm{Inv}_t\Big\}\Big\}. \tag{15}$$

3.4.2 State update with retention and symmetric round trip

Let $\eta_{\text{rt}} = 0.85$ be the round-trip efficiency and $\eta = \sqrt{\eta_{\text{rt}}}$ the one-way efficiency. With hourly retention $\rho = \exp(-1/\text{retention\_hours})$ (where `retention_hours` defaults to ∞, so $\rho = 1$),

$$\text{SOC}_t = \min\Big\{ \max\big(\rho\,\text{SOC}_{t-1} + \eta\,c_t - \tfrac{1}{\eta}d_t,\, 0\big),\, E_{\max,t}\Big\}, \tag{16}$$

$$0 \le c_t \le P_{\max,t}, \qquad 0 \le d_t \le P_{\max,t}. \tag{17}$$

Curtailment-only charging guard. When `curtailment_only_charging = TRUE`, charging is limited to otherwise excess low-carbon supply:

$$c_t \le \max\big(0,\ G_t^{\text{clean}} + I_t - D_t\big). \tag{18}$$

For transparency, the modeled SOC delta attributable to charge/discharge is

$$\Delta_t^{\text{bat}} = \eta\,c_t - \tfrac{1}{\eta}d_t. \tag{19}$$

If `allow_multiday_carry = FALSE`, $\text{SOC}_t \leftarrow 0$ at local midnight before applying retention.

3.5 System balance, curtailment, shortage

3.5.1 Energy balance

$$\sum_{g\neq\text{Fossil}} G_{g,t} + I_t + d_t + \sum_u G_{u,t} \;=\; D_t + c_t + \text{Curt}_t + \text{Short}_t. \tag{20}$$

3.5.2 Slack definitions

$$\text{Curt}_t = \max\{0,\, G_t^{\text{clean}} + I_t + \textstyle\sum_u G_{u,t} - D_t - c_t\}, \tag{21}$$

$$\text{Short}_t = \max\{0,\, D_t - G_t^{\text{clean}} - I_t - d_t - \textstyle\sum_u G_{u,t}\}. \tag{22}$$

A diagnostics residual (reported) is

$$\text{BalRes}_t = \Big(G_t^{\text{clean}} + I_t + \textstyle\sum_u G_{u,t} + d_t\Big) - \Big(D_t + c_t + \text{Curt}_t + \text{Short}_t\Big) \approx 0. \tag{23}$$

3.6 Fossil envelope and unit dispatch

3.6.1 System-level fossil requirement (pre-units)

$$R_t^{\text{fos}} = \max\big\{0,\ D_t - G_t^{\text{clean}} - I_t - d_t\big\}. \tag{24}$$

An exogenous hourly envelope $\overline{F}_t$ (from max–min fossil profiles with/without retirements) caps the legacy fleet:

$$0 \le \sum_{u\in\mathcal{U}} G_{u,t} \le \min\{\overline{F}_t,\ R_t^{\text{fos}}\}. \tag{25}$$

3.6.2 Per-unit constraints and ramping (two-pass adjustment)

Availability and min/max:

$$\chi_{u,t}\,\underline{P}_u \le G_{u,t} \le \chi_{u,t}\,\overline{P}_u. \tag{26}$$

Define the hourly ramp budget $\overline{R}_u$ (MWh/h). The algorithm enforces, after initial sampling,

$$\textbf{Backward pass:}\quad G^{\star}_{u,t-1} \;\leftarrow\; \max\big(G^{\star}_{u,t} - \overline{R}_u,\ G^{\star}_{u,t-1}\big), \tag{27}$$

$$\textbf{Forward pass:}\quad G^{\star}_{u,t} \;\leftarrow\; \max\big(G^{\star}_{u,t-1} - \overline{R}_u,\ G^{\star}_{u,t}\big), \tag{28}$$

then applies minimum output and rounding:

$$G_{u,t}^{\text{adj}} = \max\{G^{\star}_{u,t},\ \underline{P}_u\}. \tag{29}$$

3.6.3 Facility-level probabilistic sampling

Each hour's unit guidance $G^{\text{raw}}_{u,t}$ is drawn from the facility distribution by percentile p_ω:

$$G^{\text{raw}}_{u,t} = \text{Gen}^{\text{tbl}}_u(\text{DayLabel}(t), \text{Hour}(t), p_\omega). \tag{30}$$

If $\sum_u G^{\text{raw}}_{u,t} > \overline{F}_t$, proportional (or low-CF-first) reductions ensure $\sum_u G_{u,t} \le \overline{F}_t$ before the ramp passes.

3.7 Imports netting logic (headroom allocation)

Before calibration:

$$I^{\text{net}}_t = \begin{cases} I_t + \min\big(\text{Short}_t,\ \overline{I}_t - I_t\big), & \text{Short}_t > 0, \\ \min\big\{I_t,\ \max\big(0, D_t - (G^{\text{clean}}_t + \sum_u G_{u,t} + d_t)\big)\big\}, & \text{Short}_t = 0. \end{cases} \tag{31}$$

In the calibration pass (below), any extra imports required $\Delta I_t = \max(0,\, I^{\text{net,cal}}_t - I_t)$ are allocated to interties by available headroom to the maximum CF:

$$H^{\text{LT}}_{\text{HQ},t} = \text{Cap}^{\text{LT}}_{\text{HQ},t}\,\big|\overline{\text{CF}}^{\text{imp}} - \text{CF}^{\text{LT}}_{\text{HQ},t}\big|, \qquad \text{(and analogously for each intertie/channel).} \tag{32}$$

Then sequentially (long-term HQ → spot HQ → NYISO → NBSO):

$$\Delta I^{\text{LT}}_{\text{HQ},t} = \min(\Delta I_t,\ H^{\text{LT}}_{\text{HQ},t}), \quad \Delta I_t \leftarrow \Delta I_t - \Delta I^{\text{LT}}_{\text{HQ},t}, \tag{33}$$

$$\Delta I^{\text{spot}}_{\text{HQ},t} = \min(\Delta I_t,\ H^{\text{spot}}_{\text{HQ},t}), \quad \ldots \text{ (NYISO, NBSO follow).} \tag{34}$$

3.8 Emissions accounting and intensity fallback

3.8.1 Unit emissions and totals

$$E^k_{u,t} = \phi^k_u\, G^{\text{adj}}_{u,t}, \qquad k \in \{\text{CO}_2, \text{CH}_4, \text{N}_2\text{O}, \text{NO}_\text{x}, \text{SO}_2, \text{PM}_{2.5}, \text{PM}_{10}, \text{VOC}, \text{CO}\}, \tag{35}$$

$$E^k_t = \sum_u E^k_{u,t}. \tag{36}$$

3.8.2 Empirical intensity fallback (missing data)

Let $\widehat{\phi}^k_u$ be the unit's mean intensity estimated from available hours; if unavailable, fall back to the global mean $\overline{\phi}^k$ across units:

$$\phi^k_u \ \leftarrow\ \begin{cases} \widehat{\phi}^k_u, & \text{if defined,} \\ \overline{\phi}^k, & \text{otherwise.} \end{cases} \qquad E^k_{u,t} = \phi^k_u\, G^{\text{adj}}_{u,t}. \tag{37}$$

3.9 Calibration pass (recompute storage and imports)

Given finalized fossil dispatch $G^{\text{adj}}_{u,t}$, recompute battery and imports against

$$G^{\text{fos}}_t \equiv \sum_u G^{\text{adj}}_{u,t}. \tag{38}$$

Re-evaluate surplus/deficit on $(G^{\text{clean}}_t + I_t)$, rerun the storage state with the same guards and caps, and recompute imports with headroom allocation to obtain $I^{\text{net,cal}}_t$, new $\text{Curt}^{\text{cal}}_t$, $\text{Short}^{\text{cal}}_t$, and calibrated SOC/flows:

$$\text{SOC}^{\text{cal}}_t = \min\Big\{\max\big(\rho\,\text{SOC}^{\text{cal}}_{t-1} + \eta\, c^{\text{cal}}_t - \tfrac{1}{\eta} d^{\text{cal}}_t,\ 0\big),\ E_{\max,t}\Big\}, \tag{39}$$

$$\text{Curt}^{\text{cal}}_t,\ \text{Short}^{\text{cal}}_t \ge 0, \qquad I^{\text{net,cal}}_t \le \overline{I}_t. \tag{40}$$

A calibrated residual $\text{BalRes}^{\text{cal}}_t$ is reported as for the first pass.

3.10 Direct costs

3.10.1 CAPEX and fixed O&M

$$\mathrm{Cost}_y^{\mathrm{CAPEX}} = \sum_{a \in \mathcal{A}_y^+} \mathrm{CAPEX}_a, \qquad \mathrm{Cost}_y^{\mathrm{FOM}} = \sum_{a \in \mathcal{A}_y} \mathrm{FOM}_{a,y}, \tag{41}$$

where $\mathcal{A}_y^+$ are assets commissioned in y and $\mathcal{A}_y$ are operating assets in y.

3.10.2 VOM, fuel, and imports

$$\mathrm{Cost}_t^{\mathrm{VOM}} = \sum_g v_g \, G_{g,t}, \tag{42}$$

$$\mathrm{Cost}_t^{\mathrm{fuel}} = \sum_u f_u \, G_{u,t}^{\mathrm{adj}} + \sum_{g \in \{\mathrm{SMR,Biomass,Nuclear}\}} f_g \, G_{g,t}, \tag{43}$$

$$\mathrm{Cost}_t^{\mathrm{imports}} = \sum_j \kappa_j \big(I_{j,t}^{\mathrm{LT}} + I_{j,t}^{\mathrm{spot}} \big). \tag{44}$$

3.10.3 Total direct (hourly and annual)

$$\mathrm{Cost}_t^{\mathrm{direct}} = \mathrm{Cost}_t^{\mathrm{VOM}} + \mathrm{Cost}_t^{\mathrm{fuel}} + \mathrm{Cost}_t^{\mathrm{imports}}, \tag{45}$$

$$\mathrm{Cost}_y^{\mathrm{direct}} = \sum_{t \in y} \mathrm{Cost}_t^{\mathrm{direct}} + \mathrm{Cost}_y^{\mathrm{CAPEX}} + \mathrm{Cost}_y^{\mathrm{FOM}}. \tag{46}$$

3.11 Indirect costs

3.11.1 GHG externalities

$$\mathrm{Cost}_t^{\mathrm{GHG}} = s_{\mathrm{CO_2},y}^{\mathrm{GHG}} \, E_t^{\mathrm{CO_2}} + s_{\mathrm{CH_4},y}^{\mathrm{GHG}} \left(E_t^{\mathrm{CH_4}} + E_{\mathrm{res},t}^{\mathrm{CH_4}} \right) + s_{\mathrm{N_2O},y}^{\mathrm{GHG}} \, E_t^{\mathrm{N_2O}}, \tag{47}$$

with optional reservoir methane for HQ imports: $E_{\mathrm{res},t}^{\mathrm{CH_4}} = \theta_{\mathrm{res}} \, I_t^{\mathrm{HQ}}$, where θ_{res} is a reservoir methane emission factor (Delwiche et al. 2022).

3.11.2 Air pollutant damages

$$\mathrm{Cost}_t^{\mathrm{AP3}} = \sum_{k \in \{\mathrm{NO_x,SO_2,PM_{2.5},PM_{10},VOC,CO}\}} v_t^k \, E_t^k. \tag{48}$$

3.11.3 Shortage penalty (VoLL)

$$\mathrm{Cost}_t^{\mathrm{short}} = \pi_{\mathrm{short}} \cdot \mathrm{Short}_t^{\mathrm{cal}}. \tag{49}$$

3.11.4 Total indirect (hourly and annual)

$$\mathrm{Cost}_t^{\mathrm{indirect}} = \mathrm{Cost}_t^{\mathrm{GHG}} + \mathrm{Cost}_t^{\mathrm{AP3}} + \mathrm{Cost}_t^{\mathrm{short}}, \tag{50}$$

$$\mathrm{Cost}_y^{\mathrm{indirect}} = \sum_{t \in y} \mathrm{Cost}_t^{\mathrm{indirect}}. \tag{51}$$

3.12 Total system cost and NPV

$$\text{Cost}_y = \text{Cost}_y^{\text{direct}} + \text{Cost}_y^{\text{indirect}}, \tag{52}$$

$$\text{NPV} = \sum_{y \in \mathcal{Y}} \frac{\text{Cost}_y}{(1+r)^{\,y-y_0}}. \tag{53}$$

The model is evaluated under three discount rates ($r \in \{0.015, 0.020, 0.025\}$) with $y_0 = 2024$ (2024 USD).

3.13 Additional implementation notes (code parity)

- **Percentile cycling.** For each hour t, indices into percentile vectors are computed modularly to avoid re-sampling while covering solar, wind, and each import tie consistently over time.
- **Pathway-specific fossil envelope.** For pathways without scheduled retirements (A and D), $\overline{F}_t =$ `max_gen_hr_no_retirement`; otherwise $\overline{F}_t =$ `max_gen_hr_retirement`.
- **Battery daily reset (optional).** If multi-day carry is disabled, $\text{SOC}_t \leftarrow 0$ at local midnight before applying retention.
- **Diagnostics.** Report BalRes_t (and the calibrated version) and the sign guards: no simultaneous $c_t > 0$ and $d_t > 0$; no discharge when $G_t^{\text{clean}} + I_t > D_t$ under curtailment-only charging.

4 Decarbonization Pathways

4.1 Decarbonization Pathways Data Processing

The eight decarbonization pathways evaluated in this study (A, B1, B2, B3, C1, C2, C3, D) are stored in a single Excel workbook, `Decarbonization_Pathways.xlsx`, where each sheet represents one pathway and provides the annual installed capacity (MW) for every technology between 2025 and 2050. The R script `Hourly_Installed_Capacity.R` ingests this workbook and produces the hourly capacity table consumed downstream by the PHASED generation model.
The script performs the following steps:

1. **Loading libraries.** The script uses `readxl`, `data.table`, and `lubridate` for reading Excel files, efficient in-memory data manipulation, and date-time handling, respectively.
2. **Reading decarbonization data.**
   - All sheets in `Decarbonization_Pathways.xlsx` are read at once via `excel_sheets()` and a loop over `read_excel()`.
   - Each sheet is tagged with a `Pathway` column identifying the scenario.
3. **Combining data.** All per-pathway tables are concatenated row-wise into a single `data.table`.
4. **Extracting year range.** The script extracts `start_year` and `end_year` from the combined table to define the modeling horizon (2025–2050).
5. **Expansion to hourly resolution.** Annual capacities are expanded to hourly rows (8 760 or 8 784 hours per year) and exported as `Hourly_Installed_Capacity.csv`, which is read by `dispatch_curve_base_v3.R`.

Key Code Snippets:

- Loading Excel sheet names:

Listing 1: Loading Excel Sheet Names

```
sheet_names <- excel_sheets(file_path)
```

- Adding pathway metadata to each sheet:

Listing 2: Adding Pathway Metadata

```
sheet_data[, Pathway := sheet]
```

- Combining all data:

Listing 3: Combining All Sheets

```
combined_data <- rbindlist(data_tables, fill = TRUE)
```

- Extracting year range:

Listing 4: Extracting Year Range

```
years      <- unique(as.numeric(combined_data$Year))
start_year <- min(years, na.rm = TRUE)
end_year   <- max(years, na.rm = TRUE)
```

4.2 Pathway Capacity and Retirement Visualizations

The headline pathway figure of the manuscript (installed 2050 capacity per pathway, with the planned fossil-fuel retirement schedule) is produced by the Jupyter notebook `installed_capacity_retirement_schedule.ipynb` (Python). The notebook:

1. Loads `Decarbonization_Pathways.xlsx` (all sheets) and stacks the per-pathway DataFrames.
2. Loads the existing fossil-fuel facility data (`Fossil_Fuel_Facilities_Data.csv`) and computes the active fossil capacity year-by-year (2025–2050) by filtering on `Retirement_year`.
3. Merges the active fossil capacity time series back onto the pathway table to form the "Old fossil" column. For pathway A (no retirements), the 2025 value is held constant.
4. Computes per-pathway capacity *differences* relative to pathway A in 2050 for each technology (Old fossil, Nuclear, Hydropower, Biomass, Imports QC/NYISO/NBSO, Onshore/Offshore Wind, Solar, New NG, SMR), and renders a stacked horizontal bar chart with a hatched "status quo" baseline.
5. Produces a companion stacked-bar figure of the existing fossil fleet capacity in 2025 versus the remaining fleet in 2050 (after planned retirements), broken down by primary fuel type (Pipeline Natural Gas, Coal, Diesel Oil, Other Oil, Residual Oil, Wood).

Outputs: `Decarbonization_pathways.svg`, `Decarbonization_pathways_retirements.svg`, and `retirements.csv`. This script, together with `Hourly_Installed_Capacity.R`, ensures that the decarbonization pathways data is integrated and organized consistently for both the model run and the figure-generation pipeline.

5 PHASED Generation Model

5.1 Wind and Solar Hourly Capacity Factors

5.1.1 Overview

The `Wind_Solar_Hourly_CF.R` script processes hourly wind and solar generation data to derive percentile-mapped hourly capacity factors used by the PHASED dispatch curve (Section on Stochastic availability sampling in the Model Formulation). The script standardizes the input data and aligns it with the time zones and calendar conventions used in the model.

5.1.2 Detailed Explanation

1. **Loading data.** Hourly generation data are read from CSV files (`solar_CF.csv`, `onwind_CF.csv`, `offwind_CF.csv`), and the required columns are selected.
2. **Time zone adjustment.** Timestamps are converted from UTC to Eastern Standard Time (EST), aligning with the local conditions used in ISO-NE demand data.

3. **Handling leap years.** A leap-year adjustment ensures consistency by mapping day-of-year 366 to 365 in non-leap years, avoiding errors in models that depend on a 365-day calendar.

4. **Percentile binning.** Hourly capacity factors are organized by `DayLabel` (1–365) and `Hour` (1–24) and tabulated into 99 percentile bins for the Monte Carlo sampler.

5.1.3 Key Code

Listing 5: Adjusting timestamps and handling leap years

```r
data_processing_func <- function(file, columns_to_keep) {
  data <- readr::read_csv(file, show_col_types = FALSE)
  data <- data %>% dplyr::select(all_of(columns_to_keep))
  data$Date <- as.POSIXct(data$Date, format = "%m/%d/%Y %I:%M:%S %p", tz = "UTC")

  # Convert UTC to EST
  data$Date <- data$Date - hours(5)

  # Add DayLabel and Hour columns
  data <- data %>%
    mutate(DayLabel = yday(Date), Hour = hour(Date) + 1)

  # Adjust DayLabel for non-leap years
  if (!is_leap_year(year(data$Date[365]))) {
    data <- data %>% mutate(DayLabel = ifelse(DayLabel == 366, 365, DayLabel))
  }
}
```

5.2 Small Modular Reactors (SMR)

5.2.1 Overview

The `SMR_CF.R` script defines the operational characteristics of small modular reactors used in pathways that include new SMR build-out. These include:

- Capacity factor (90%, treated as a constant baseload).
- Ramp rate and minimum power output.
- Core fuel characteristics, including enrichment and burnup.

5.2.2 Key Code

Listing 6: Defining SMR Specifications

```r
data <- data.frame(
  Label                    = "SMR300",
  Category                 = "SMR",
  Nameplate_Capacity_MW    = 300,
  CF                       = 0.9,
  Ramp                     = 1,
  Minimum_Power_Output_MW  = 60,
  Enrichment               = "4.95%",
  Fuel                     = "UO2",
  Reactor_Lifetime         = 60
)
```

5.3 Fossil Fuels: Facility Data and Emissions

5.3.1 Fossil Fuel Facility Data

The `Fossil_Fuels_Facilities_Data.R` script processes historical fossil fuel facility data for New England (CT, ME, MA, NH, RI, VT), drawn from the U.S. EPA eGRID and EIA Form 860/923 datasets. This data

captures the existing capabilities and limitations of the regional fossil fuel infrastructure, including planned retirements.

Detailed Explanation:

1. **Loading data.** Facility data for all U.S. states is loaded, including facility location, capacity, operating status, ramp rates, primary fuel type, and retirement year.
2. **Filtering for active facilities.** Facilities are filtered to include only those located in New England and those that are operational (i.e., not retired as of the modeling start year, 2025).
3. **Ramp rate conversion.** Ramping designators (e.g., `"10M"`, `"1H"`, `"12H"`, `"OVER"`) are converted into numeric values in hours. This standardization ensures compatibility with the per-unit ramp budget $\overline{R}_u$ used in the dispatch curve.

Key Code Excerpts:

Listing 7: Ramp Rate Conversion Function

```r
convert_ramp_to_numeric <- function(ramp) {
  if (is.na(ramp)) {
    return(24)         # Default to 24 hours for missing values
  } else if (ramp == "10M") {
    return(0.1666667) # 10 minutes
  } else if (ramp == "1H") {
    return(1)          # 1 hour
  } else if (ramp == "12H") {
    return(12)         # 12 hours
  } else if (ramp == "OVER") {
    return(24)         # Default for 'OVER'
  } else {
    return(24)
  }
}
```

Listing 8: Filtering Facilities for New England

```r
# Filter data for active facilities in New England
new_england_states <- c("CT", "ME", "MA", "NH", "RI", "VT")
facilities_data_NE <- facilities_data[State %in% new_england_states]
facilities_data_NE <- facilities_data_NE[
  !grepl("retired", Operating_Status, ignore.case = TRUE)
]
```

5.3.2 Fossil Fuel Generation and Emissions

The `Fossil_Fuels_Gen_Emissions.R` script compiles hourly generation and emissions data for fossil fuel facilities in New England from the U.S. EPA CAMPD (Clean Air Markets Program Data) database. This dataset underpins the per-unit hourly generation distributions $\text{Gen}_u^{\text{tbl}}$ used by the percentile sampler and the empirical emission factors ϕ_u^k used in the emissions module.

Detailed Explanation:

1. **Reading emissions data.** Hourly emissions data is read for each New England state.
2. **Combining emissions data.** Data from individual states is combined into a single dataset.
3. **Cross-referencing with facility data.** Emissions data is filtered to include only active facilities, ensuring consistency with the facility dataset and with the retirement schedule used by the dispatch curve.

Key Code Excerpts:

Listing 9: Combining Emissions Data

```
combined_emissions_data <- rbindlist(all_emissions_data, fill = TRUE)

# Filter emissions data to match active facilities
active_facilities <- facilities_data_NE$Facility_Unit.ID
combined_emissions_data <- combined_emissions_data[
  Facility_Unit.ID %in% active_facilities
]
```

5.4 New Fossil Fuel Facilities

5.4.1 Overview

The `Fossil_Fuels_New.R` script models the addition of new natural-gas combined-cycle (NGCC) facilities to meet future energy demands in pathways that include new fossil build-out. These facilities are phased in starting from 2033, with three units per facility and one facility added annually.

Detailed Explanation:

1. **Defining facility specifications.** Each facility is given a unique ID (`NGCC3000`, `NGCC3001`, …), and the year it comes online is specified.

2. **Expanding facility data.** Each facility is expanded into three units to represent its operational characteristics (e.g., ramp and minimum-output behavior) accurately.

Key Code Excerpts:

Listing 10: Defining New Fossil Fuel Facilities

```
# Define new facilities starting in 2033 (one per year)
new_facilities <- data.table(
  Facility_ID = paste0("NGCC", 3000:(3000 + num_facilities - 1)),
  Year_Online = start_year:(start_year + num_facilities - 1)
)

# Expand each facility into 3 units
new_facility_units <- new_facilities[, .(
  Unit_ID     = 1:3,
  Facility_ID,
  Year_Online
)]
```

5.5 Imports

5.5.1 Overview

The `imports.R` script processes historical electricity imports into ISO-NE and constructs the hourly capacity-factor tables used by the PHASED model to sample import variability under uncertainty. Key components include:

- Loading daily imports data for the years 2011 to 2023 from an Excel workbook.
- Setting the maximum capacity factor for transmission lines to 95%, consistent with NREL ATB guidance.
- Computing percentiles for electricity imports over the historical period to populate the 99-percentile sampling table used by the Monte Carlo sampler.

5.5.2 Detailed Explanation

1. **Loading libraries.** The script uses `dplyr`, `readxl`, `lubridate`, `data.table`, and `tidyverse` for data manipulation, reading Excel files, and date handling.

2. **Defining time range.** The imports data is analyzed for the period 1 January 2011 to 31 December 2023.
3. **Loading data.**
   - The script identifies and reads sheets from the Excel file corresponding to years between 2011 and 2023.
   - Data from each sheet is combined into a single dataset, appending a `Year` column for clarity.
4. **Percentile calculations.** The `calculate_percentiles()` function returns 99 percentile values (1% to 99%) for a specified column of the imports dataset. The resulting percentile table is the empirical distribution used by the PHASED Monte Carlo sampler to draw $\mathrm{CF}_{j,t}^{\mathrm{spot}}(\omega)$ for each intertie.

5.5.3 Key Code

Listing 11: Defining Time Range and Max Capacity Factor

```
start_date                <- as.Date("2011-01-01")
end_date                  <- as.Date("2023-12-31")
max_CF_transmission_lines <- 0.95  # According to NREL ATB
```

The imports data is loaded from an Excel file, extracting only the sheets corresponding to the years of interest (2011–2023):

Listing 12: Loading Electricity Imports Data

```
file <- "daily_capacity_status.xlsx"
imports_data <- data.frame()
years        <- as.character(2011:2023)
sheet_names  <- excel_sheets(file)
sheet_names_of_interest <- sheet_names[
  sapply(sheet_names, function(name) name %in% years)
]

for (sheet_name in sheet_names_of_interest) {
  sheet_data       <- read_excel(file, sheet = sheet_name)
  sheet_data$Year  <- sheet_name
  imports_data     <- rbind(imports_data, sheet_data)
}
```

Listing 13: Percentile Calculation Function

```
calculate_percentiles <- function(data, column) {
  quantiles <- quantile(
    data[[column]],
    probs  = seq(0.01, 0.99, by = 0.01),
    na.rm  = TRUE
  )
  tibble(Percentile = seq(1, 99), Value = quantiles)
}
```

This function is applied per intertie (Hydro-Québec, NYISO, NBSO) to produce the `Imports_CF.csv` table consumed by `dispatch_curve_base_v3.R`.

5.5.4 Insights and Results

The `imports.R` script prepares the data for modeling electricity imports in PHASED, providing:

- A unified dataset of daily electricity imports from 2011 to 2023.
- An empirical 99-percentile distribution of import capacity factors per intertie, used as the basis for stochastic sampling.
- Capacity factor assumptions ($\overline{\mathrm{CF}}^{\mathrm{imp}} = 0.95$) for transmission infrastructure.

5.6 Commercial Battery Storage

5.6.1 Overview

The PHASED dispatch curve treats commercial battery storage as a symmetric round-trip device with explicit inverter, duration, and retention constraints. Storage parameters are not estimated from a separate script; rather, they are configured in the runtime block of `dispatch_curve_base_v3.R` and applied consistently in both the initial dispatch pass and the calibration pass.

5.6.2 Configured Parameters

The defaults used in this study are summarized in Table 5.6.2 below and correspond to the `cfg` list at the top of `dispatch_curve_base_v3.R`.

Parameter	Value	R variable
Round-trip efficiency η_{rt}	0.85	`cfg$rt_eff`
One-way efficiency $\eta = \sqrt{\eta_{\text{rt}}}$	≈ 0.922	derived
Storage duration H^{dur} (h)	8	`cfg$duration_hours`
Inverter cap column (if present)	`Inverter_MW`	`cfg$inverter_col`
Grid charging permitted	FALSE	`cfg$allow_grid_charging`
Curtailment-only charging	TRUE	`cfg$curtailment_only_charging`
Hourly retention via `retention_hours`	∞ ($\rho = 1$)	`cfg$retention_hours`
Multi-day SOC carry	TRUE	`cfg$allow_multiday_carry`

5.6.3 Power cap from inverter and duration

For each hour t, given the nameplate energy capacity $E_{\max,t}$ (column `Storage_MW` in `Hourly_Installed_Capacity`) and the configured duration H^{dur}, the derived power limit is

$$P_{\max,t}^{\text{derived}} \;=\; \frac{E_{\max,t}}{H^{\text{dur}}}, \tag{54}$$

and the effective hourly power cap combines this with the optional inverter limit Inv_t:

$$P_{\max,t} \;=\; \max\{0,\; \min(P_{\max,t}^{\text{derived}},\; \text{Inv}_t)\}. \tag{55}$$

If `Inverter_MW` is absent from the hourly capacity table, the derived limit alone is used.

5.6.4 State update with retention and symmetric round trip

With one-way efficiency η and hourly retention $\rho = \exp(-1/\texttt{retention\_hours})$, the state of charge is updated as

$$\text{SOC}_t = \min\Big\{ \max\big(\rho\,\text{SOC}_{t-1} + \eta\, c_t - \tfrac{1}{\eta} d_t,\; 0\big),\; E_{\max,t} \Big\}, \tag{56}$$

$$0 \le c_t \le P_{\max,t}, \qquad 0 \le d_t \le P_{\max,t}. \tag{57}$$

When `allow_multiday_carry = FALSE` the algorithm sets $\text{SOC}_t \leftarrow 0$ at local midnight before applying retention. With `retention_hours = Inf` (the default), $\rho = 1$ and there is no passive self-discharge.

5.6.5 Curtailment-only charging guard

The battery may only absorb energy from otherwise-curtailed low-carbon supply when `curtailment_only_charging = TRUE`:

$$c_t \;\le\; \max\big(0,\; G_t^{\text{clean}} + I_t - D_t\big), \tag{58}$$

where G_t^{clean} is total non-fossil generation, I_t is total imports, and D_t is hourly demand. This guard prevents the battery from acting as an arbitrage device against fossil generation in the absence of a price model (`allow_grid_charging = FALSE`).

5.6.6 Diagnostics

For transparency, the modeled SOC delta attributable to charge/discharge is recorded as

$$\Delta_t^{\text{bat}} \;=\; \eta\, c_t - \tfrac{1}{\eta} d_t, \tag{59}$$

and is written to the hourly results as `Battery_flow`. Two sign-guard warnings are issued if violated:

- No simultaneous charge and discharge: $c_t > 0$ *and* $d_t > 0$ never occurs.
- No discharge when surplus low-carbon supply is positive: $d_t = 0$ whenever $G_t^{\text{clean}} + I_t > D_t$ under curtailment-only charging.

5.6.7 Calibration pass

After the fossil dispatch is finalized in `dispatch_curve_base_v3.R`, the entire storage block is rerun against the calibrated demand-and-supply balance to produce `Calibrated_Storage_status`, `Calibrated_Battery_charge_grid`, and `Calibrated_Battery_discharge_grid`. These calibrated quantities feed the annual system summaries in the dispatch-curve results processing step and the downstream cost calculations.

5.6.8 Key Code

Listing 14: Battery configuration (`cfg`) and SOC update loop

```r
## Runtime configuration (battery + run notes)
cfg <- list(
  rt_eff                  = 0.85,          # round-trip efficiency (symmetric)
  duration_hours          = 8,             # energy-to-power duration (h)
  inverter_col            = "Inverter_MW", # optional column in Hourly_Installed_Capacity
  allow_grid_charging     = FALSE,         # no price model here
  curtailment_only_charging = TRUE,        # charge only from excess low-carbon supply
  retention_hours         = Inf,           # SOC_t = SOC_{t-1} * exp(-1/retention_hours)
  allow_multiday_carry    = TRUE,          # if FALSE, SOC resets at midnight
  high_penalty_unserved   = 10000          # documentation only
)

# Derived efficiencies and limits
eta <- sqrt(cfg$rt_eff)
dispatch_data[, derived_power_limit := Storage_MW / cfg$duration_hours]
if (cfg$inverter_col %in% names(dispatch_data)) {
  dispatch_data[, inverter_limit_MW :=
    pmin(get(cfg$inverter_col), derived_power_limit, na.rm = TRUE)]
} else {
  dispatch_data[, inverter_limit_MW := derived_power_limit]
}
dispatch_data[, battery_power_limit := pmax(0, inverter_limit_MW)]

# Hourly SOC update (curtailment-only charging guard applied inside loop)
for (i in seq_len(nrow(dispatch_data))) {
  cap_MWh <- dispatch_data$Storage_MW[i]
  p_lim   <- dispatch_data$battery_power_limit[i]
  # ... compute c_t, d_t respecting guards, then:
  soc[i] <- min(max(rho * soc[i-1] + eta * c_t - d_t / eta, 0), cap_MWh)
}
```

5.7 Demand

5.7.1 Overview

The script `demand.R` ingests hourly ISO-NE demand data and conditions it for use in the PHASED dispatch curve. Two operations are performed:

- Estimating missing February 29 demand values in leap years by averaging the February 28 and March 1 values for each hour.

- Cleaning and standardizing the hourly demand series so that calendar conventions match those used elsewhere in the model (DayLabel 1–365 in non-leap years, with leap-day rows filled in).

The output, an hourly demand series D_t over the modeling horizon (2025–2050), is one of the primary inputs to `dispatch_curve_base_v3.R`.

5.7.2 Leap Year Adjustment

The script checks for leap years and fills missing February 29 demand by averaging February 28 and March 1 values, hour by hour. The following code performs this operation:

Listing 15: Filling Missing Leap Year Data

```r
fill_leap_year_data <- function(data) {
  for (year in unique(data$Year)) {
    if (year %% 4 == 0 && (year %% 100 != 0 || year %% 400 == 0)) {  # Check if leap year
      for (hour in 0:23) {
        feb_28_data <- data[Year == year & Month == 2 & Day == 28 & Hour == hour, Demand]
        mar_1_data  <- data[Year == year & Month == 3 & Day == 1  & Hour == hour, Demand]

        if (length(feb_28_data) == 1 && length(mar_1_data) == 1) {
          leap_day_demand <- mean(c(feb_28_data, mar_1_data), na.rm = TRUE)
          data <- rbind(data, data.table(
            Year   = year,
            Month  = 2,
            Day    = 29,
            Hour   = hour,
            Demand = leap_day_demand
          ))
        }
      }
    }
  }
  return(data)
}
```

5.8 Randomization

5.8.1 Overview

The `randomization.R` script generates the pseudo-random percentile sequences that drive the probabilistic sampling of capacity factors and facility-level generation in the PHASED dispatch curve. Each sequence corresponds to one Monte Carlo draw and indexes percentiles between 1 and 99 of the empirical hourly distributions. The script:

- Defines the number of sequences and the length of each sequence.
- Uses a small helper function to generate the sequences with `sample()`.
- Saves the generated sequences as a CSV that is loaded once by the dispatch model.

5.8.2 Detailed Explanation

1. **Loading libraries.** The `randtoolbox` library is loaded; while the current implementation uses base R's `sample()`, `randtoolbox` provides quasi-random alternatives that can be substituted if needed.
2. **Defining parameters.**
   - `num_sequences = 1e6`: the total number of Monte Carlo sequences (one per simulation).
   - `num_numbers = 250`: the length of each sequence (250 integers in $\{1, \ldots, 99\}$).

   Together these parameters define the scope and granularity of the randomization. During dispatch, indices into the percentile vector are computed modularly so that solar, wind, and each import tie are sampled consistently over the modeling horizon.

3. **Generating pseudo-random sequences.** A custom function, `generate_pseudo_random_sequences()`, generates each sequence by sampling integers from 1 to 99 with replacement.

4. **Saving the random sequences.** The matrix of sequences is transposed (so that columns index simulations and rows index positions within a sequence) and written to `Random_Sequence.csv` for subsequent use by the PHASED model.

5.8.3 Key Code

Listing 16: Pseudo-Random Sequence Generator

```r
generate_pseudo_random_sequences <- function(num_sequences, num_numbers) {
  random_sequences <- replicate(
    num_sequences,
    sample(1:99, num_numbers, replace = TRUE),
    simplify = FALSE
  )
  return(random_sequences)
}
```

Listing 17: Generating and Transposing Random Sequences

```r
# Parameters
num_sequences <- 1e6
num_numbers   <- 250

# Generate pseudo-random sequences
Percentile_sequences_random <- generate_pseudo_random_sequences(
  num_sequences, num_numbers
)

# Convert the list to a matrix and transpose it so that columns index simulations
Percentile_sequences_matrix <- t(do.call(rbind, Percentile_sequences_random))
```

Listing 18: Saving Random Sequences to CSV

```r
file_path_random_csv <- "Random_Sequence.csv"
write.csv(Percentile_sequences_matrix, file_path_random_csv, row.names = FALSE)
```

5.8.4 Insights and Applications

The randomization step enables:

- Creation of large-scale probabilistic datasets that can be reused across discount-rate runs and pathways without re-sampling.
- Consistent representation of uncertainty and variability across the wind, solar, fossil unit, and import-tie distributions.
- Efficient on-disk storage so that simulations are reproducible bit-for-bit when the same row of `Random_Sequence.csv` is loaded.

The generated pseudo-random sequences are consumed by `dispatch_curve_base_v3.R` (the PHASED dispatch curve) and propagate uncertainty through both the generation model and the downstream cost and impact calculations.

5.9 Dispatch Curve

The PHASED dispatch curve is implemented in `dispatch_curve_base_v3.R`. Once the core dispatch routines have produced hourly system-level outputs and per-unit fossil-fuel outputs, end-to-end simulations are run across all eight decarbonization pathways, the results are aggregated, exported, and validated against deterministic ISO-NE Roadmap trajectories.

5.9.1 Simulation Execution and Aggregation

For each Monte Carlo simulation index and each pathway, the script calls in turn:

1. `dispatch_curve(sim, pathway)` to compute the clean-plus-storage dispatch (low-carbon generation, imports, and battery flows).
2. `dispatch_curve_adjustments()` to sample and adjust fossil-fuel outputs under per-unit ramp constraints (the two-pass backward/forward ramp algorithm).
3. `dispatch_curve_calibrations()` to recombine the adjusted fossil outputs and re-run the battery and imports blocks (the calibration pass).

Each function returns a `data.table` tagged with `Simulation` and `Pathway`. The nested results are flattened into two master tables: one for the final hourly dispatch results and one for the facility-level fossil fuel outputs.

Listing 19: Running simulations and combining results

```r
# 1. Define simulations and pathways
pathways      <- unique(Hourly_Installed_Capacity$Pathway)
n_simulations <- 1   # set higher for full Monte Carlo runs

# 2. Execute each sim and pathway
simulation_results <- lapply(1:n_simulations, function(sim) {
  lapply(pathways, function(pathway) {
    dc_results   <- dispatch_curve(sim, pathway)
    ff_adjusted  <- dispatch_curve_adjustments(dc_results)
    final_hourly <- dispatch_curve_calibrations(dc_results, ff_adjusted)
    # Tag with sim and pathway
    dc_results$Simulation  <- ff_adjusted$Simulation  <- final_hourly$Simulation  <- sim
    dc_results$Pathway     <- ff_adjusted$Pathway     <- final_hourly$Pathway     <- pathway
    list(final_hourly = final_hourly, fossil_fuels = ff_adjusted)
  })
})

# 3. Flatten into two data.tables
combined_final_hourly_results <- do.call(rbind, lapply(simulation_results, function(s) {
  do.call(rbind, lapply(s, `[[`, "final_hourly"))
}))
combined_fossil_fuels_results <- do.call(rbind, lapply(simulation_results, function(s) {
  do.call(rbind, lapply(s, `[[`, "fossil_fuels"))
}))
```

5.9.2 Exporting Results

After aggregation, the two tables are written to CSV for downstream analysis and figure generation:

Listing 20: Writing aggregated results to CSV

```r
# Save combined hourly dispatch results
write.csv(combined_final_hourly_results,
          "/path/to/Hourly_Results_NE.csv",
          row.names = FALSE)

# Save combined facility-level fossil outputs
write.csv(combined_fossil_fuels_results,
          "/path/to/Facility_Level_Results.csv",
          row.names = FALSE)
```

5.9.3 Validation against Deterministic ISO-NE Trajectories

To confirm that the dispatch-curve implementation reproduces the intended aggregate dynamics, the model is validated against deterministic ISO-NE trajectories from the Massachusetts 2050 Decarbonization Roadmap

workbook. The validation is performed by `Validation.R`. Specifically, for a given decarbonization pathway and Roadmap scenario (for example `B1` under `High Electrification`), annual total load and renewable generation from PHASED are compared against the Roadmap benchmarks for years 2025–2050 in five-year steps. The corresponding figure is SI Figure S4 of the manuscript.

The yearly summary output from PHASED (`Yearly_Results.csv`) and an external Excel workbook containing the ISO-NE benchmark trajectories are used as inputs. Paths and scenario metadata are set at the top of the validation script:

Listing 21: Validation settings and input files

```
target_scenario   <- "High Electrification"
target_pathway    <- "B1"
years_to_validate <- seq(2025, 2050, by = 5)

proj_dir   <- "/path/to/project/root"
model_file <- file.path(proj_dir, "2 Generation Expansion Model",
                        "5 Dispatch Curve", "4 Final Results",
                        "1 Comprehensive Days Summary Results",
                        "Yearly_Results.csv")
excel_file <- file.path(proj_dir, "4 External Data",
                        "Massachusetts 2050 Decarbonization Roadmap Study",
                        "Massachusetts Workbook of Energy Modeling Results 2024.xlsx")
```

Model-side annual metrics. From the yearly model output, total annual load and total annual renewables are constructed for each simulation and year. Total load is defined to be consistent with the Roadmap accounting (all generation including imports, with no curtailment subtraction):

Listing 22: Annual model aggregates

```
model_data <- fread(model_file)[
  Pathway == target_pathway & Year %in% years_to_validate
]

model_sims <- model_data[, .(
  Renewables = Solar_TWh + Onshore_TWh + Offshore_TWh +
               Hydro_TWh + Biomass_TWh,
  Total_Load = (Old_Fossil_Fuels_adj_TWh + New_Fossil_Fuel_TWh) +
               Nuclear_TWh +
               (Solar_TWh + Onshore_TWh + Offshore_TWh +
                Hydro_TWh + Biomass_TWh) +
               Calibrated_Total_import_net_TWh
), by = .(Simulation, Year)]

model_stats <- model_sims[, .(
  Model_Renewables = median(Renewables),
  Model_TotalLoad  = median(Total_Load)
), by = Year]
```

Benchmark trajectories from the Roadmap workbook. The benchmark series are extracted from the Roadmap Excel workbook (sheet `"10. Electricity Generation"`). For each validation year the script locates the appropriate scenario-year column, sums the renewable rows (solar, wind, hydro), and reconstructs total generation by adding nuclear, imports via transmission, and fossil generation (gas, coal, oil):

Listing 23: Benchmark extraction (conceptual)

```
raw_gen <- as.data.table(
  read_excel(excel_file, sheet = "10. Electricity Generation",
             col_names = FALSE)
)

get_benchmark <- function(yr) {
  # identify column for target_scenario and yr
  # read clean generation summary block
  # sum renewables (solar, wind, hydro)
```

```
  # read fossil rows from top section (gas, coal, oil)
  # add nuclear and transmission imports
  data.table(
    Year             = yr,
    Bench_Renewables = renew_val,
    Bench_TotalLoad  = total_val
  )
}

bench_results <- rbindlist(
  lapply(years_to_validate, get_benchmark)
)
```

Goodness of fit and plots. The model median series is merged with the benchmark series, and coefficient of determination (R^2) statistics are computed for both total load and renewables:

Listing 24: R^2 calculation

```
validation_df <- merge(model_stats, bench_results, by = "Year")

calc_r2 <- function(obs, pred) {
  ss_res <- sum((obs - pred)^2)
  ss_tot <- sum((obs - mean(obs))^2)
  1 - ss_res / ss_tot
}

r2_load  <- calc_r2(validation_df$Bench_TotalLoad,
                    validation_df$Model_TotalLoad)
r2_renew <- calc_r2(validation_df$Bench_Renewables,
                    validation_df$Model_Renewables)
```

Finally, a diagnostic figure is generated with two panels (one for total annual load and one for renewable generation). Each panel shows the median model trajectory against the Roadmap benchmark, with the corresponding R^2 value annotated:

Listing 25: Diagnostic plotting

```
plot_data <- rbind(
  melt(validation_df[, .(
         Year, Model = Model_TotalLoad,
         Benchmark = Bench_TotalLoad)],
       id.vars = "Year")[, Metric := "Total Annual Load"],
  melt(validation_df[, .(
         Year, Model = Model_Renewables,
         Benchmark = Bench_Renewables)],
       id.vars = "Year")[, Metric := "Renewable Generation"]
)

ggplot(plot_data, aes(Year, value, color = variable)) +
  geom_line() + geom_point() +
  facet_wrap(~Metric, scales = "free_y") +
  theme_minimal()
```

These validation statistics and plots provide a compact check that the PHASED dispatch module reproduces the intended deterministic Roadmap trajectories for both total energy demand and renewable generation across the modeling horizon.

6 Dispatch Curve Results Processing

6.1 Overview

After PHASED produces hourly dispatch and fossil-fuel facility outputs (`dispatch_curve_base_v3.R`), two post-processing steps are applied:

1. **Spatial enrichment:** each fossil facility is assigned to its county (and state) via a spatial join against U.S. Census county geometries, with Connecticut planning regions remapped to the legacy county

nomenclature where needed.

2. **Annual system summaries:** key system metrics—renewable penetration, battery discharge, shortages, and CO_2 emissions—are computed by year, simulation, and pathway and combined across all scenarios into a single tidy table.

6.2 Spatial Enrichment of Facility Results

1. Read the facility-level summary CSV into a `data.table` and drop any phantom index columns.
2. Filter out rows missing latitude/longitude and convert to an `sf` object (WGS84, CRS 4326).
3. Load county geometries (via `tigris::counties()` plus a New England GeoJSON), ensuring both layers use CRS 4326.
4. Spatially join facilities to counties with `st_within`, then:
   - For Connecticut, map current planning-region names back to historical county names and assign the corresponding GEOIDs (the Census switched CT reporting units in 2022).
   - Manually correct any unmatched facility IDs (e.g. `10823_S42`, `10823_S43` to Suffolk, MA).
5. Save the enriched table (with `County_State`, `GEOID`, etc.) to CSV.

6.3 Annual System Summaries

1. For each hourly-results CSV, compute:

$$\text{RenewablePenetration} = \frac{\text{Solar\_MWh} + \text{Onshore\_MWh} + \text{Offshore\_MWh}}{\begin{array}{c}\text{Clean\_MWh+Calibrated\_Total\_import\_net\_MWh} \\ + \text{Old\_Fossil\_Fuels\_adj\_MWh+New\_Fossil\_Fuel\_MWh}\end{array}},$$

$$\text{BatteryDischarges\_GWh} = \sum(\text{Calibrated\_Battery\_discharge\_grid})/10^3,$$

$$\text{Shortages\_TWh} = \sum(\text{Calibrated\_Shortage\_MWh})/10^6,$$

$$\text{CO2\_tons} = \sum(\text{CO2\_tons}).$$

2. Summarize by `Year`, `Simulation`, and `Pathway`.
3. Row-bind across all files and write `Battery_Data.csv`. This file is the primary input to several downstream Python notebooks (e.g. `Energy_Storage.ipynb`).

6.4 Key Code Snippets

Listing 26: Spatial join and county assignment

```r
#-- Load and clean facility results
Yearly_Fac <- fread(file_path)[, V1 := NULL]
fac_clean  <- Yearly_Fac[!is.na(latitude) & !is.na(longitude)]
fac_sf     <- st_as_sf(fac_clean, coords = c("longitude","latitude"), crs = 4326)

#-- Load counties, spatial join
counties <- counties(cb = TRUE, class = "sf") %>% st_transform(4326)
fac_sf   <- st_join(fac_sf, counties, join = st_within)

#-- Fix CT planning regions, county names and GEOIDs
fac_dt <- as.data.table(fac_sf)
fac_dt[STUSPS == "CT", NAME  := planning_to_county[NAME]]
fac_dt[STUSPS == "CT", GEOID := county_to_geoid[NAME]]

#-- Manual fixes for known mismatches ...
fwrite(fac_dt, file.path(output_path,
       "Yearly_Facility_Level_Results_County_added_in.csv"))
```

Listing 27: Annual summary of renewables, storage, shortages, CO_2

```
#--- Read all hourly outputs, compute per-file summaries
files        <- list.files(folder_path, "^Hourly_Results_.*\\.csv$", full.names = TRUE)
summary_list <- list()
for (f in files) {
  dt <- fread(f)
  dt[, Renewable.Penetration :=
       (Solar_MWh + Onshore_MWh + Offshore_MWh) /
       (Calibrated_Total_import_net_MWh + Old_Fossil_Fuels_adj_MWh +
        Clean_MWh + New_Fossil_Fuel_MWh)]
  summary_list[[f]] <- dt[, .(
    RP_mean                = mean(Renewable.Penetration, na.rm = TRUE),
    RP_max                 = max( Renewable.Penetration, na.rm = TRUE),
    RP_min                 = min( Renewable.Penetration, na.rm = TRUE),
    Battery_Discharges_GWh = sum(Calibrated_Battery_discharge_grid, na.rm = TRUE) / 1e3,
    Shortages_TWh          = sum(Calibrated_Shortage_MWh,           na.rm = TRUE) / 1e6,
    CO2_tons               = sum(CO2_tons,                          na.rm = TRUE)
  ), by = .(Year, Simulation, Pathway)]
}
combined_summary <- rbindlist(summary_list)
write.csv(combined_summary,
          file.path(output_path, "Battery_Data.csv"),
          row.names = FALSE)
```

7 Total Costs

7.1 CAPEX, FOM, and VOM Costs

7.1.1 CAPEX and FOM for Fossil Energy

The `1_CAPEX_FOM_ATB_Fossil.R` script calculates capital expenditures (CAPEX) and fixed operation and maintenance costs (FOM) for fossil-fuel generators using NREL ATB cost data. It:

- Loads NREL ATB (Annual Technology Baseline) cost data for fossil-fuel technologies.
- Extracts data for coal, natural gas (combined-cycle and combustion turbine), oil, and wood.
- Computes net present value (NPV) of CAPEX and FOM streams. The script is re-run for each of the three discount rates considered in the manuscript (1.5%, 2%, and 2.5%); the example below shows $r = 0.02$.

Key Code:

Listing 28: Calculating NPV for Fossil CAPEX and FOM

```
calculate_npv <- function(dt, rate, base_year) {
  npv <- sum(dt[, 2] / (1 + rate)^(dt[, 1] - base_year))
  return(npv)
}

discount_rate <- 0.02   # run also at 0.015 and 0.025
base_year     <- 2023

Coal_NPC <- Fossil_Fuels_NPC[
  grepl("Coal", Primary.Fuel.Type, ignore.case = TRUE)
]
Gas_CC_NPC <- Fossil_Fuels_NPC[
  grepl("Gas",      Primary.Fuel.Type, ignore.case = TRUE) &
  grepl("Combined", Unit.Type,         ignore.case = TRUE)
]
```

7.1.2 CAPEX and FOM for Non-Fossil Energy

The `2_CAPEX_FOM_ATB_Non_Fossil.R` script handles renewable and nuclear technologies. It:

- Loads NREL ATB data for non-fossil technologies, including solar, onshore wind, offshore wind, large nuclear, SMRs, and hydropower.
- Computes CAPEX and FOM for each technology across the three ATB cost scenarios (Advanced, Moderate, Conservative).

Key Code:

Listing 29: Loading and Setting Non-Fossil Data

```
ATBe <- fread("ATBe.csv")
path <- "Installed Capacity and CF Non Fossil"
files <- list.files(path = path, pattern = "\\.csv$", full.names = TRUE)

datasets <- list()
for (file in files) {
  dataset_name <- gsub(".*/|\\.csv$", "", file)
  datasets[[dataset_name]] <- fread(file)
}

list2env(datasets, envir = .GlobalEnv)  # release datasets into the environment
setDT(Solar_NPC)
setDT(Nuclear_hydro_bio_NPC)
```

7.1.3 CAPEX and FOM for Imports

The `3_CAPEX_FOM_Imports.R` script calculates CAPEX and FOM costs for new transmission/import capacity (Hydro-Québec interties under pathway B3, and other inter-regional infrastructure). It:

- Loads import capacity data across three cost scenarios (S1 / Advanced, S2 / Moderate, S3 / Conservative).
- Computes the year-on-year new capacity built and applies unit CAPEX/FOM costs.

Key Code:

Listing 30: Loading and Calculating Import Capacity

```
Imports_S1 <- read_excel("Import Capacity.xlsx", sheet = "S1")
Imports_S2 <- read_excel("Import Capacity.xlsx", sheet = "S2")
Imports_S3 <- read_excel("Import Capacity.xlsx", sheet = "S3")

Imports_Capacity <- rbindlist(
  list(Imports_S1, Imports_S2, Imports_S3),
  use.names = TRUE, fill = TRUE
)
Imports_Capacity[, new_capacity := QC - shift(QC, 1, type = "lag"),
                 by = Scenario]
```

7.1.4 VOM for Fossil Energy

The `4_VOM_ATB_Fossil.R` script calculates variable operation and maintenance (VOM) costs for fossil fuels:

- Merges fossil-fuel generation data with facility-specific attributes (unit type, primary fuel type).
- Multiplies generation by per-unit VOM rates from the NREL ATB, by ATB cost scenario.

Key Code:

Listing 31: Calculating VOM Costs for Fossil Fuels

```
Yearly_Facility_Level_Results <- selected_cols[
  Yearly_Facility_Level_Results, on = "Facility_Unit.ID"
]
```

```
  Yearly_Facility_Level_Results[
    , VOM_Cost := Generation * VOM_Rate, by = Unit.Type
  ]
```

7.1.5 VOM for Non-Fossil Energy

The `5_VOM_ATB_Non_Fossil.R` script calculates VOM costs for renewable and nuclear energy. It:

- Processes VOM costs for solar, onshore/offshore wind, hydropower, large nuclear, and SMRs.
- Filters by ATB scenario and technology-specific parameters (`Technology`, `TechDetail`).

Key Code:

Listing 32: Processing VOM Costs for Non-Fossil Technologies

```
process_non_fossil <- function(technology, techdetail, dataset,
                               column_name, simulation, scenario) {
  dataset <- dataset[
    Technology == technology & TechDetail == techdetail
  ]
  dataset[, VOM_Cost := Generation * VOM_Rate]
  return(dataset)
}
```

7.1.6 Insights and Applications

Together, these scripts provide comprehensive CAPEX, FOM, and VOM cost assessments across fossil, non-fossil, and import infrastructure. The outputs are consumed by `13_Total_Costs_and_Uncertainty_Analysis.R` and feed both the per-simulation cost table and the across-simulation summary used in the main-text figures.

7.2 Fuel Costs

7.2.1 Fuel Costs for Fossil Energy

The `7_Fuel_ATB_Fossil.R` script calculates fuel costs for fossil-fuel generators using the NREL ATB dataset. Key operations include:

- Loading Consumer Price Index (CPI) data via the `fredr` API to adjust historical costs to 2024 USD.
- Loading NREL ATB fuel cost data for coal, natural gas, and oil.
- Calculating the Net Present Value (NPV) of fuel costs. The script is re-run for each of the three discount rates considered in the manuscript (1.5%, 2%, and 2.5%); the example below shows $r = 0.02$.

Key Code:

Listing 33: Loading and Adjusting Fuel Costs for Fossil Fuels

```
# Set discount rate and base year
discount_rate <- 0.02   # run also at 0.015 and 0.025
base_year     <- 2023

# Load CPI data to calculate conversion rate
# (FRED API key should be supplied via environment variable, e.g.
#  fredr_set_key(Sys.getenv("FRED_API_KEY")) )
cpi_data <- fredr(series_id        = "CPIAUCSL",
                  observation_start = as.Date("2000-01-01"),
                  observation_end   = as.Date("2024-01-01"))
cpi_2021 <- filter(cpi_data, year(date) == 2021) %>% summarise(YearlyAvg = mean(value))
cpi_2024 <- filter(cpi_data, year(date) == 2024) %>% summarise(YearlyAvg = mean(value))
conversion_rate <- cpi_2024$YearlyAvg / cpi_2021$YearlyAvg

# Load ATB fuel cost data
```

```
file_path      <- "ATB_2021_Fuel_Costs.csv"
ATB_Fuel_Costs <- fread(file_path)
ATB_Fuel_Costs[, Adjusted_Cost := Original_Cost * conversion_rate]
```

This script adjusts fossil-fuel costs to 2024 USD and calculates NPVs to evaluate long-term fuel expenditure.

7.2.2 Fuel Costs for Non-Fossil Energy

The `8_Fuel_ATB_Non_Fossil.R` script processes fuel costs for non-fossil energy sources such as large nuclear, SMRs, biomass, and any hydropower that incurs a per-MWh fueling charge. Its primary functions are to:

- Load NREL ATB cost data for non-fossil fuel types.
- Filter by `Technology`, `TechDetail`, and ATB scenario.
- Compute per-MWh fueling cost streams to be combined with generation in the dispatch results.

Key Code:

Listing 34: Processing Non-Fossil Fuel Costs

```
process_non_fossil <- function(technology, techdetail, dataset,
                               column_name, simulation, scenario) {
  dataset <- dataset[
    Technology == technology & TechDetail == techdetail
  ]
  dataset[, Adjusted_Cost := Fuel_Cost * VOM_Rate]
  return(dataset)
}

# Example: processing nuclear fuel costs
process_non_fossil("Nuclear", "Advanced", ATBe,
                   "Fuel_Cost", "Simulation1", "Scenario1")
```

This script enables scenario-based analysis of non-fossil fuel costs under varying economic and technological assumptions.

7.2.3 Insights and Applications

These scripts together provide:

- Fossil-fuel costs adjusted to 2024 USD via CPI rebasing.
- Non-fossil fuel costs integrated into the scenario sweep over ATB cost categories.

The combined fuel-cost outputs feed `13_Total_Costs_and_Uncertainty_Analysis.R`, which assembles them into the per-simulation total-cost table.

7.3 Imports Costs

7.3.1 Overview

The `6_Imports.R` script calculates costs related to electricity imports into ISO-NE. Key functionalities include:

- Adjusting historical import prices to 2024 USD using Consumer Price Index (CPI) data retrieved from the `fredr` API.
- Calculating NPV of imports costs. The script is re-run for each of the three discount rates considered in the manuscript (1.5%, 2%, and 2.5%); the example below shows $r = 0.02$.
- Processing capacity data across multiple cost scenarios (S1 / Advanced, S2 / Moderate, S3 / Conservative) to estimate annual costs.

7.3.2 Detailed Explanation

1. **Loading CPI data.**
   - The script retrieves CPI data and computes conversion rates from the relevant historical year (e.g., 2021) to 2024 USD.
   - Conversion rates are calculated as ratios of yearly-average CPI values.
2. **Loading capacity data.**
   - Import capacity data is loaded from an Excel file, with separate sheets for each scenario (S1, S2, S3).
   - Year-on-year new capacity additions are computed within each scenario.
3. **NPV calculation.** The script calculates the NPV of costs for each scenario, providing a long-term cost assessment.

7.3.3 Key Code

CPI adjustments:

Listing 35: Loading and Adjusting CPI Data

```r
# Set FRED API key via environment variable, e.g.
#   fredr_set_key(Sys.getenv("FRED_API_KEY"))
cpi_data <- fredr(series_id         = "CPIAUCSL",
                  observation_start = as.Date("2000-01-01"),
                  observation_end   = as.Date("2024-01-01"))

# Calculate conversion rate from 2021 USD to 2024 USD
cpi_2021 <- filter(cpi_data, year(date) == 2021) %>% summarise(YearlyAvg = mean(value))
cpi_2024 <- filter(cpi_data, year(date) == 2024) %>% summarise(YearlyAvg = mean(value))
conversion_rate_2021 <- cpi_2024$YearlyAvg / cpi_2021$YearlyAvg
```

Loading capacity data:

Listing 36: Loading Import Capacity Data

```r
path <- "Imports_Capacity.xlsx"

# Load data for different scenarios
Imports_S1 <- read_excel(path, sheet = "S1")
Imports_S2 <- read_excel(path, sheet = "S2")
Imports_S3 <- read_excel(path, sheet = "S3")

# Combine and calculate new capacity per year, per scenario
Imports_Capacity <- rbindlist(
  list(Imports_S1, Imports_S2, Imports_S3),
  use.names = TRUE, fill = TRUE
)
Imports_Capacity[, new_capacity := QC - shift(QC, 1, type = "lag"),
                 by = Scenario]
```

NPV calculation:

Listing 37: Calculating NPV for Import Costs

```r
calculate_npv <- function(dt, rate, base_year, col) {
  npv <- sum(dt[[col]] / (1 + rate)^(dt[["Year"]] - base_year))
  return(npv)
}

discount_rate <- 0.02   # run also at 0.015 and 0.025
base_year     <- 2023

# Example NPV calculation for scenario S1
```

```
npv_scenario1 <- calculate_npv(
  Imports_Capacity[Scenario == "S1"],
  discount_rate, base_year, "new_capacity"
)
```

7.3.4 Insights and Applications

This script provides a detailed assessment of import costs:

- Inflation adjustment ensures that all costs are expressed in 2024 USD.
- Scenario-based analysis enables comparison across import-cost trajectories.
- NPV calculations provide a long-term perspective on import-related expenditures and feed the consolidated cost table assembled by `13_Total_Costs_and_Uncertainty_Analysis.R`.

7.4 GHG Emissions Costs

7.4.1 Overview

The `9_GHG_Emissions.R` script calculates the social cost of greenhouse-gas emissions from PHASED's fossil-fuel dispatch. Key functionalities include:

- Loading annual GHG emissions data in metric tons CO_2-equivalent (IPCC AR4 GWP).
- Interpolating social costs for CO_2, CH_4, and N_2O from 2025 to 2050.
- Adjusting those costs to 2024 USD via CPI data from the `fredr` API.
- Calculating NPV of total GHG costs. The script is re-run for each of the three discount rates considered in the manuscript (1.5%, 2%, and 2.5%); the example below shows $r = 0.025$.

7.4.2 Detailed Explanation

1. **Loading emissions data.** Read facility-level and system-level CSVs of annual GHG emissions (all in metric tons CO_2-eq).
2. **CPI adjustment.** Fetch the CPI series (2000–2024) via `fredr`, compute the 2020 $\rightarrow$ 2024 conversion factor, and scale all future SCC endpoint values.
3. **Cost interpolation.** Define SCC endpoints in 2025 and 2050 for CO_2, CH_4, and N_2O; linearly interpolate for each year between 2025 and 2050.
4. **NPV calculation.** For each simulation and pathway, sum annual

$$\text{Cost}_t = \text{CO}_{2,t} \times c_{\text{CO}_2,t} \; + \; \text{CH}_{4,t} \times c_{\text{CH}_4,t} \; + \; \text{N}_2\text{O}_t \times c_{\text{N}_2\text{O},t}$$

and discount from 2025 to 2050:

$$\text{NPV} = \sum_{t=2025}^{2050} \frac{\text{Cost}_t}{(1+r)^{\,t-2024}}.$$

7.4.3 Key Code

CPI and cost curve:

```r
discount_rate <- 0.025   # run also at 0.015 and 0.02
base_year     <- 2024

# Set FRED API key via environment variable, e.g.
#   fredr_set_key(Sys.getenv("FRED_API_KEY"))
cpi <- fredr("CPIAUCSL", as.Date("2000-01-01"), as.Date("2024-01-01"))
f20 <- mean(filter(cpi, year(date) == 2020)$value)
f24 <- mean(filter(cpi, year(date) == 2024)$value)
rate <- f24 / f20

cost25 <- list(CO2 = 130, CH4 = 1590, N2O = 39972) * rate
cost50 <- list(CO2 = 205, CH4 = 3547, N2O = 65635) * rate

interp_cost <- function(y, start = 2025, end = 2050, c0, c1) {
  c0 + (c1 - c0) * (y - start) / (end - start)
}
GHG_Costs <- data.table(Year = 2025:2050)[
   , `:=`(
     CO2_cost = interp_cost(Year, c0 = cost25$CO2, c1 = cost50$CO2),
     CH4_cost = interp_cost(Year, c0 = cost25$CH4, c1 = cost50$CH4),
     N2O_cost = interp_cost(Year, c0 = cost25$N2O, c1 = cost50$N2O)
   )]
```

NPV calculation:

```r
# after merging annual total_CO2, total_CH4_eq, total_N2O_eq and GHG_Costs:
dt[, annual_cost := total_CO2 * CO2_cost +
                    total_CH4_eq * CH4_cost +
                    total_N2O_eq * N2O_cost]

calc_npv <- function(d) {
  sum(d$annual_cost / (1 + discount_rate)^(d$Year - base_year))
}

# per simulation and pathway
npv_list <- dt[, .(NPV = calc_npv(.SD)), by = .(Simulation, Pathway)]

fwrite(npv_list[, .(
  NPV_min  = min(NPV),
  NPV_mean = mean(NPV),
  NPV_max  = max(NPV)
), by = Pathway],
   "GHG_Emissions.csv")

fwrite(npv_list, "GHG_Emissions_per_simulation.csv")
```

7.5 Air Pollutant Emissions Costs

7.5.1 Overview

The `10_Air_emissions.R` script calculates costs associated with air pollutant emissions (NO_x, SO_2, $PM_{2.5}$, PM_{10}, VOC, CO) from PHASED's fossil-fuel dispatch, using county-specific marginal damage estimates from the AP3 model (Muller, 2022). Key functionalities include:

- Converting emission quantities to metric tons for consistency.
- Adjusting marginal damage estimates to 2024 USD using Consumer Price Index (CPI) data from the `fredr` API.
- Calculating NPV of air-pollutant costs. The script is re-run for each of the three discount rates considered in the manuscript (1.5%, 2%, and 2.5%); the example below shows $r = 0.02$.

7.5.2 Detailed Explanation

1. **Emission unit conversions.**

- Emission data are converted from pounds to U.S. tons, and subsequently to metric tons (for consistency with international standards and the AP3 marginal damages).

2. **CPI adjustments.** Per-pollutant marginal damages are adjusted to 2024 USD using the ratio of CPI values between the AP3 reference year and 2024.

3. **NPV calculation.** The script calculates the NPV of air-pollutant costs, enabling a long-term economic comparison across pathways and discount rates.

7.5.3 Key Code

Unit conversions:

Listing 38: Converting Emission Units to Metric Tons

```
lbs_tons_conversion <- 1 / 2204.62  # lbs to U.S. tons
ton_conversion      <- 0.907185     # U.S. tons to metric tons

# Example conversion
emission_data[, Metric_Tons := Emissions_Lbs * lbs_tons_conversion * ton_conversion]
```

CPI adjustments:

Listing 39: Adjusting Costs Using CPI Data

```
# Set FRED API key via environment variable, e.g.
#   fredr_set_key(Sys.getenv("FRED_API_KEY"))
cpi_data <- fredr(series_id         = "CPIAUCSL",
                  observation_start = as.Date("2000-01-01"),
                  observation_end   = as.Date("2024-01-01"))

# Calculate conversion rate from 2000 USD to 2024 USD
cpi_2000 <- filter(cpi_data, year(date) == 2000) %>% summarise(YearlyAvg = mean(value))
cpi_2024 <- filter(cpi_data, year(date) == 2024) %>% summarise(YearlyAvg = mean(value))
conversion_rate <- cpi_2024$YearlyAvg / cpi_2000$YearlyAvg

# Adjust costs
emission_data[, Adjusted_Cost := Original_Cost * conversion_rate]
```

NPV calculation:

Listing 40: Calculating NPV for Air Pollutant Costs

```
calculate_npv <- function(dt, rate, base_year, col) {
  npv <- sum(dt[[col]] / (1 + rate)^(dt[["Year"]] - base_year))
  return(npv)
}

discount_rate <- 0.02   # run also at 0.015 and 0.025
base_year     <- 2024

# Example NPV calculation
npv_air_emissions <- calculate_npv(
  emission_data, discount_rate, base_year, "Adjusted_Cost"
)
```

7.5.4 Insights and Applications

This script supports air-pollutant cost modeling by:

- Ensuring consistency in emission quantities via unit conversions.
- Adjusting historical marginal damages to 2024 USD for accurate comparisons.
- Providing a long-term economic perspective through NPV calculations.

The output is consumed by `13_Total_Costs_and_Uncertainty_Analysis.R` as the `Air_emissions_*_bUSD` block of the consolidated cost table, and also feeds the county-level emissions choropleth plotted in `Emissions.ipynb`.

7.6 Unmet Demand Penalty Costs

7.6.1 Overview

The R script `11_Unmet_Demand.R` translates hours of unmet demand from the PHASED dispatch into discounted dollar penalties using ISO-NE-aligned value-of-lost-load (VoLL) guidance. Specifically, it:

- Loads annual shortage results from `Yearly_Results_Shortages.csv`.
- Converts unmet demand (MWh) into dollar penalties using a VoLL of $3,500/MWh in 2024 USD (consistent with the manuscript's Methods section and SI Figure S2's pay-for-performance and value-of-lost-load curves).
- Computes NPVs of those penalties, discounting from a 2024 base year at the configured rate (1.5%, 2%, or 2.5% depending on the run).
- Exports per-simulation NPVs and pathway-level summaries to CSV for inclusion in the total-cost tabulation.

7.6.2 Detailed Explanation

1. **Load shortage data.** Read `Yearly_Results_Shortages.csv` (annual unmet demand in MWh, by Year, Simulation, and Pathway) into a `data.table`.
2. **Compute dollar penalties.**

$$\text{Unmet\_Demand\_USD\_total}_y = \text{Unmet\_Demand\_total\_MWh}_y \times 3500$$

   with the constant 3,500 USD/MWh expressed in 2024 USD.

3. **NPV calculation.** For each (Simulation, Pathway) pair, define

$$\text{NPV} = \sum_{y=2025}^{2050} \frac{\text{Unmet\_Demand\_USD\_total}_y}{(1+r)^{\,y-2024}}$$

   where r is the run-specific discount rate (the example below uses $r = 0.025$; the same script is rerun with $r \in \{0.015, 0.020, 0.025\}$ for the three discount-rate scenarios).

4. **Export.** Write the per-simulation NPVs to `Unmet_demand.csv`, dropping rows with zero NPV (i.e. simulations where every year had zero unmet demand). The downstream R Code: `13_Total_Costs_and_Uncertainty_Analysis.R` consumes this file as one of the cost components.

7.6.3 Key Code

Listing 41: Unmet-demand penalty NPV

```r
library(data.table)

# NPV helper
calculate_npv <- function(dt, rate, base_year, col) {
  sum(dt[[col]] / (1 + rate)^(dt$Year - base_year))
}

discount_rate <- 0.025
base_year     <- 2024

# VoLL in USD/MWh (2024 USD), per ISO-NE guidance
```

```r
VoLL_2024 <- 3500

# Load shortage results
file_path      <- "Yearly_Results_Shortages.csv"
output_path    <- ".../Total_Costs_Results"
Yearly_Results <- fread(file_path)

# Compute annual dollar penalties
Yearly_Results[, Unmet_Demand_USD_total := Unmet_Demand_total_MWh * VoLL_2024]

# Per-simulation, per-pathway NPV
npv_list <- list()
for (sim in unique(Yearly_Results$Simulation)) {
  for (path in unique(Yearly_Results$Pathway)) {
    dt  <- Yearly_Results[Simulation == sim & Pathway == path]
    npv <- calculate_npv(dt, discount_rate, base_year, "Unmet_Demand_USD_total")
    if (npv != 0) {
      npv_list[[paste(sim, path)]] <- data.table(
        Simulation = sim, Pathway = path, NPV = npv
      )
    }
  }
}
combined_npvs <- rbindlist(npv_list)

# Summary by pathway (in billions of 2024 USD)
combined_npvs[, .(
  mean_NPV_bUSD = mean(NPV) / 1e9,
  sd_NPV_bUSD   = sd(NPV)   / 1e9
), by = Pathway]

# Save results
fwrite(combined_npvs, file.path(output_path, "Unmet_demand.csv"))
```

7.7 Beyond the Fence Line Costs

7.7.1 Overview

The scripts `12_Other_S3_CAN_Hydro_CostAssumptions.R` and `12_Other_S3_CAN_Hydro.R` compute the "beyond-the-fence-line" costs of pathway B3, which expands long-term hydropower imports from Québec. They:

- Read the decarbonization pathways to compute Québec import-capacity differences between B3 and B1.
- Derive annual new hydropower build (ΔMW) from the cumulative import-capacity difference.
- Fetch CPI (2016–2024) to convert the ISO-NE CAPEX assumption ($5,537/kW) and the ATB cost ranges into 2024 USD.
- Compute annual CAPEX, fixed O&M (FOM), and variable O&M (VOM) costs for the new Canadian hydropower required to support B3.
- Estimate reservoir CH_4 emissions associated with the additional imports using the Delwiche et al. (2022) emission factor, project a CH_4 social-cost curve (2025–2050), and compute the corresponding NPV. The script is re-run for each of the three discount rates considered in the manuscript (1.5%, 2%, and 2.5%); the example below shows $r = 0.025$.

7.7.2 Hydropower Capacity Build

- QC capacity difference: $\Delta_{\mathrm{QC}}(t) = \mathrm{cumsum}\big[\mathrm{QC}_{\mathrm{B3}}(t) - \mathrm{QC}_{\mathrm{B1}}(t)\big]$.
- Base hydro capacity: $H(t) = \Delta_{\mathrm{QC}}(t)/\mathrm{CF}$.
- Year-on-year build: $\mathrm{Build}(t) = \max\{0,\ H(t) - H(t-1)\}$.

7.7.3 Cost Calculations

- CPI rate: $r_{\text{CPI}} = \text{CPI}_{2024}/\text{CPI}_{2016}$.
- Unit CAPEX: $\text{CAPEX}_{\text{unit}} = \$5{,}537/\text{kW} \times 1000 \times r_{\text{CPI}}$.
- FOM rate: $\{1.5\%, 2.5\%\} \times \text{CAPEX}_{\text{unit}}$ per year.
- VOM: \$0.58/MWh.
- Annual costs:

$$\begin{aligned} C_{\text{CAPEX}}(t) &= \text{Build}(t) \times \text{CAPEX}_{\text{unit}}, \\ C_{\text{FOM}}(t) &= \text{QC\_cap\_MW}(t) \times \text{FOM\_rate/CF}, \\ C_{\text{VOM}}(t) &= \text{QC\_MWh}(t) \times \text{VOM/CF}. \end{aligned}$$

7.7.4 CH_4 Emissions and NPV

- CH_4 emissions (tonnes): $\text{Imports\_QC\_TWh} \times \text{EF} \times \frac{16}{12} \times 10^{-3}$, where EF is the Delwiche et al. (2022) reservoir CH_4 emission factor (low/high range used to bracket uncertainty).
- A CH_4 social-cost curve is projected from 2025 to 2050 via CPI-adjusted endpoints and linear interpolation.
- NPV at discount rate r (1.5%, 2%, or 2.5% depending on the run):

$$\text{NPV} = \sum_{t=2025}^{2050} \frac{C_t}{(1+r)^{\,t-2024}}.$$

- Export CAPEX, FOM, VOM, and CH_4 NPVs by pathway to CSV. In the consolidated cost table assembled by `13_Total_Costs_and_Uncertainty_Analysis.R`, these are collected under the `CAN_CAPEX_*`, `CAN_FOM_*`, `CAN_FOM_*` (VOM is folded into FOM in the final table), and `CAN_CH4_*` columns. They are zeroed for B3(1) and retained (with imports zeroed) for B3(2) in the sensitivity split described in the Total Costs section.

7.8 Total Costs and Uncertainty Analysis

7.8.1 Overview

The R script `13_Total_Costs_and_Uncertainty_Analysis.R` assembles every cost element produced by the per-component scripts (CAPEX/FOM, VOM, fuel, imports, GHG, air emissions, unmet demand, and Canada-side hydropower CAPEX/FOM/VOM and CH_4) into two final tables:

- `All_Costs_per_Simulation.csv`: per-simulation, per-pathway NPVs for every cost component, with total cost and B1-difference columns appended.
- `All_Costs.csv`: across-simulation means by pathway and cost type (CAPEX, Fixed O&M, Variable O&M, Fuel, Imports, GHG emissions, Air emissions, Unmet demand penalty, and Sum), after dropping outlier simulations.

A companion script, `14_Tabulate_All_Results.R`, takes `All_Costs.csv` for each of the three discount rates (1.5%, 2%, 2.5%) and writes a formatted Excel matrix (`Formatted_Cost_Matrix_Xpct.xlsx`) with mean and bracketed min–max for every (pathway, cost type) pair.

7.8.2 Detailed Explanation

1. **Load component CSVs.** List all `*.csv` files in the cost-results folder, read each with `fread()`, and assign each table to a global object named after its file.

2. **Summarize each cost category.**

   - *CAPEX & FOM*: bind the fossil (`CAPEX_Fixed_Fossil`), non-fossil (`CAPEX_Fixed_Non_Fossil`), and imports (`CAPEX_FOM_Imports`) tables, group by `Pathway` (and ATB cost-category scenario where applicable), and sum NPVs. Compute `_mean_bUSD`, `_max_bUSD`, and `_min_bUSD` across cost categories.
   - *VOM, Fuel, Imports, GHG, Air, Unmet*: analogous per-component summaries.
   - *Canada side (B3 only)*: aggregate hydropower CAPEX/FOM/VOM and CH_4 NPVs, combine with the adjusted QC imports tables, and broadcast across all simulations. Other pathways receive zeros in the `CAN_*` columns.

3. **Merge all components.** Full-join the per-component sim-level tables on `(Simulation, Pathway)` using `purrr::reduce(., full_join)` and drop incomplete rows.

4. **Split B3 into B3(1) and B3(2).** The B3 pathway is duplicated to support a sensitivity reading of the Canada-side accounting:

   - `B3(1)` retains imports costs but zeroes all `CAN_*` columns (i.e. ignores Canada-side externalities).
   - `B3(2)` retains `CAN_*` columns but zeroes the `Imports_*` columns (i.e. attributes all Canadian costs to the importing region rather than splitting between import payments and beyond-the-fence-line damages).

   The original `B3` rows are then removed.

5. **Compute totals.** Sum every `_mean_bUSD`, `_max_bUSD`, and `_min_bUSD` column to produce Total Costs Mean in bUSD, etc. Drop any simulation that does not have results for all pathways (so that B1-differences are well defined).

6. **B1 differences.** For each simulation, subtract the corresponding B1 total from every other pathway's total to produce `B1_Diff_mean_bUSD`, `B1_Diff_max_bUSD`, and `B1_Diff_min_bUSD`.

7. **Filter outliers and aggregate.** Pivot to long form. For each pathway, compute total cost per simulation, flag simulations outside $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$, drop those simulations from all rows, then average by (Pathway, Cost_Type, Statistic).

8. **Save CSVs.** Write `All_Costs_per_Simulation.csv` (per-simulation detail) and `All_Costs.csv` (aggregated means by pathway and cost type) to the discount-rate-specific output folder.

7.8.3 Key Code

Listing 42: Core consolidation steps (`13_Total_Costs_and_Uncertainty_Analysis.R`)

```
# 1. Read all cost CSVs in the results folder
csv_files <- list.files(folder_path, pattern = "\\.csv$",
                        full.names = TRUE, recursive = FALSE)
data_tables <- lapply(csv_files, function(file) {
  dt   <- fread(file)
  name <- tools::file_path_sans_ext(basename(file))
  assign(name, dt, envir = .GlobalEnv)
  dt
})

# 2. Summarize CAPEX & FOM across fossil, non-fossil, and imports
total_CAPEX_FOM <- bind_rows(total_CAPEX_FOM_Fossil,
                             total_CAPEX_Fixed_Non_Fossil,
```

```
                                total_CAPEX_FOM_Imports) %>%
  group_by(Pathway, Cost_Category) %>%
  summarize(FOM_bUSD = sum(FOM)/billion,
            CAPEX_bUSD = sum(CAPEX)/billion, .groups = "drop") %>%
  group_by(Pathway) %>%
  summarize(across(c(FOM_bUSD, CAPEX_bUSD),
                   list(mean = mean, max = max, min = min),
                   .names = "{.col}_{.fn}"), .groups = "drop")

# 3. Other costs summarized similarly (VOM, Fuel, Imports, GHG, Air, Unmet, CAN)

# 4. Merge everything on (Simulation, Pathway)
sim_lists <- list(CAPEX = total_CAPEX, FOM = total_FOM, VOM = total_VOM,
                  Fuel = total_Fuel, Imports = total_Imports,
                  GHG = total_GHG_Emissions, Air = total_Air_Emissions,
                  Unmet = total_Unmet_demand, CAN = total_CAN_full)
All_Costs <- purrr::reduce(sim_lists, full_join, by = c("Simulation","Pathway"))
All_Costs <- All_Costs[complete.cases(All_Costs), ]

# 5. Split B3 -> B3(1) and B3(2)
B3_base <- All_Costs %>% filter(Pathway == "B3")
B3_1_rows <- B3_base %>%
  mutate(Pathway = "B3(1)", across(starts_with("CAN_"),      ¬ 0))
B3_2_rows <- B3_base %>%
  mutate(Pathway = "B3(2)", across(starts_with("Imports"), ¬ 0))
All_Costs_final <- All_Costs %>%
  filter(Pathway != "B3") %>%
  bind_rows(B3_1_rows, B3_2_rows) %>%
  arrange(Simulation, Pathway)

# 6. Total costs and B1-differences per simulation
All_Costs_final <- All_Costs_final %>%
  mutate(Total_Costs_mean_bUSD = rowSums(select(., ends_with("_mean_bUSD")), na.rm = TRUE),
         Total_Costs_max_bUSD  = rowSums(select(., ends_with("_max_bUSD")),  na.rm = TRUE),
         Total_Costs_min_bUSD  = rowSums(select(., ends_with("_min_bUSD")),  na.rm = TRUE)) ...
           %>%
  group_by(Simulation) %>%
  mutate(B1_Diff_mean_bUSD = Total_Costs_mean_bUSD - Total_Costs_mean_bUSD[Pathway == "B1"],
         B1_Diff_max_bUSD  = Total_Costs_max_bUSD  - Total_Costs_max_bUSD[ Pathway == "B1"],
         B1_Diff_min_bUSD  = Total_Costs_min_bUSD  - Total_Costs_min_bUSD[ Pathway == ...
             "B1"]) %>%
  ungroup()

write.csv(All_Costs_final,
          file.path(output_path, "All_Costs_per_Simulation.csv"),
          row.names = FALSE)

# 7. Drop outlier simulations (1.5 x IQR per pathway) and aggregate
outlier_sims <- long_data %>%
  group_by(Pathway, Simulation) %>%
  summarise(total_cost = sum(Cost_bUSD, na.rm = TRUE), .groups = "drop") %>%
  group_by(Pathway) %>%
  mutate(Q1 = quantile(total_cost, 0.25),
         Q3 = quantile(total_cost, 0.75),
         IQR = Q3 - Q1,
         lower = Q1 - 1.5 * IQR,
         upper = Q3 + 1.5 * IQR) %>%
  filter(total_cost < lower | total_cost > upper) %>%
  distinct(Simulation) %>% pull(Simulation)

combined_ranges <- long_data %>%
  filter(!Simulation %in% outlier_sims) %>%
  group_by(Pathway, Cost_Type, Statistic) %>%
  summarise(Cost_bUSD = mean(Cost_bUSD, na.rm = TRUE), .groups = "drop")

write.csv(combined_ranges,
          file.path(output_path, "All_Costs.csv"),
          row.names = FALSE)
```

7.8.4 Correlated Uncertainty Propagation and Global Sensitivity

The same script (and its Python companion `Uncertainty_analysis.ipynb`) implements the correlated-uncertainty and global-sensitivity analyses that are central to the manuscript. These analyses operate on `All_Costs_per_Simulation.csv` pooled across the three discount rates (1.5%, 2%, 2.5%).

Spearman correlation among cost drivers. The cost-component columns (CAPEX, FOM, VOM, Fuel, Imports, GHG, Air, Unmet demand, plus the discount rate as an explicit input variable) are correlated pairwise using Spearman's ρ, then hierarchically clustered (`average` linkage, Euclidean distance on the correlation rows). Non-significant cells ($p > 0.05$) are marked with a $\times$ in the rendered heatmap. The resulting figure is `Fig_SI1_Correlation_Spearman_Clustered_LightDiag_NoCbar.svg` (with a companion skinny colorbar SVG).

Global sensitivity via permutation importance. For the pooled dataset, a random-forest regressor (`n_estimators=400`, `min_samples_leaf=2`) is fitted with cost components plus discount rate as features and `Total_Costs_mean_bUSD` as the target. Permutation importance is computed on a held-out test set (`test_size=0.25`); bootstrap resamples produce normalized 95% CIs on the importance of each driver. The same procedure is repeated per pathway to produce the grouped-bar figure.

Gaussian-copula triangular sampling. For the joint correlated uncertainty propagation of fuel prices, CAPEX, and discount rate, the notebook implements a Gaussian-copula construction that maps uniform marginals through triangular per-row distributions (with low/mode/high pulled from the per-simulation cost tables) while preserving a target Spearman correlation matrix. The discount rate is sampled from the discrete pool $\{0.015, 0.020, 0.025\}$ in this construction.

7.8.5 Insights and Applications

This pipeline:

- Ensures every cost driver is on the same footing and traceable back to its source CSV.
- Produces both granular (per-simulation) and summary (mean by pathway) cost tables, with B3 explicitly split into B3(1) and B3(2) to disentangle in-region versus beyond-the-fence-line Canadian costs.
- Identifies and removes simulation outliers (1.5 $\times$ IQR per pathway) for robust scenario comparison.
- Produces the inputs for both the headline stacked-bar/error-bar figure (`Total_Costs.ipynb`) and the SI sensitivity figures (`Uncertainty_analysis.ipynb`).

7.9 CPI and Other Cost Variables

7.9.1 CPI for electricity (BLS API)

The `CPI_Electricity.R` script retrieves Consumer Price Index (CPI) data from the U.S. Bureau of Labor Statistics (BLS) API. It focuses on CPI series related to electricity costs across multiple regions used in benchmarking. Key functionalities include:

- Retrieving regional CPI data for electricity using the BLS API.
- Structuring the retrieved data into a tabular format.
- Storing the processed data for integration into cost modeling workflows.

Detailed Explanation:

1. **API integration.** The script uses the `blsAPI` and `rjson` libraries to query the BLS API for CPI series data.

2. **Regions and series.** The script specifies electricity-related series IDs and their corresponding regions (e.g., New England, Boston, Middle Atlantic).

3. **Data processing.** Retrieved data is organized into a data frame with fields for year, period, and CPI values.

Key Code:

Listing 43: Retrieving CPI Data Using BLS API

```r
library(blsAPI)
library(rjson)

# Read BLS registration key from environment variable rather than hard-coding
api_key    <- Sys.getenv("BLS_API_KEY")
series_ids <- c("APU011072610", "APUS11A72610",
                "APU012072610", "APUS12A72610", "APUS12B72610")
regions    <- c("New England", "Boston", "Middle Atlantic",
                "New York", "Philadelphia")

payload <- list(
  'seriesid'        = series_ids,
  'regions'         = regions,
  'startyear'       = '2022',
  'endyear'         = '2024',
  'registrationkey' = api_key
)

response  <- blsAPI(payload)
json_data <- fromJSON(response)
```

Listing 44: Organizing CPI Data into a Data Frame

```r
cpi_data <- do.call(rbind, lapply(json_data$Results$series, function(series) {
  data.frame(
    seriesID = series$seriesID,
    Region   = series$regions,
    year     = sapply(series$data, function(x) x$year),
    period   = sapply(series$data, function(x) x$period),
    value    = sapply(series$data, function(x) as.numeric(x$value))
  )
}))
```

7.9.2 Supplemental costs calculation

The `Supplemental_Costs_calc.R` script calculates supplemental costs using GDP, population, and household data. Key functionalities include:

- Loading and filtering population and GDP growth data for the years 2025–2050.
- Converting GDP and household data to 2023 USD using CPI adjustments.
- Preparing the processed data for integration into demand and cost models.

Detailed Explanation:

1. **Data loading.** Population and GDP data are loaded from an Excel file and filtered for the target years (2025–2050).

2. **CPI adjustments.** CPI data is retrieved from the FRED API and used to rebase GDP values from 2012 to 2023 USD.

3. **Data preparation.** The processed data is stored for further use in supplemental cost modeling.

Key Code:

Listing 45: Loading Population and GDP Data

```r
path <- "/path/to/population_GDP_growth.xlsx"

household_data <- read_excel(path)
household_data <- household_data %>%
  filter(Year ≥ 2025 & Year ≤ 2050)
colnames(household_data) <- c("Year", "n_households_millions", "GDP_bUSD_2012")
```

Listing 46: Adjusting GDP to 2023 USD

```r
library(fredr)

# Set FRED API key via environment variable, e.g.
#   fredr_set_key(Sys.getenv("FRED_API_KEY"))
cpi_data <- fredr(series_id         = "CPIAUCSL",
                  observation_start = as.Date("2000-01-01"),
                  observation_end   = as.Date("2024-01-01"))

cpi_2012 <- filter(cpi_data, year(date) == 2012) %>% summarise(YearlyAvg = mean(value))
cpi_2023 <- filter(cpi_data, year(date) == 2023) %>% summarise(YearlyAvg = mean(value))

conversion_rate <- cpi_2023$YearlyAvg / cpi_2012$YearlyAvg
household_data$GDP_bUSD_2023 <- household_data$GDP_bUSD_2012 * conversion_rate
```

7.9.3 Insights and Applications

These scripts provide critical data-processing capabilities:

- **`CPI_Electricity.R`**: Supports cost modeling by providing up-to-date CPI data for electricity across key regions.
- **`Supplemental_Costs_calc.R`**: Prepares supplemental cost data (GDP and household), adjusted for inflation to 2023 USD.

The outputs are essential for integrating economic and demographic variables into the energy modeling framework.

API keys. All scripts that previously hard-coded FRED or BLS API keys in their source have been migrated to read them from environment variables (`FRED_API_KEY`, `BLS_API_KEY`). Users reproducing the pipeline should request their own keys from FRED and BLS and export them before running the scripts.

8 Non-monetized Ecological Impacts

This section documents four Python (Jupyter) notebooks that quantify non-monetized ecological impacts—land-use change, avian mortality, water withdrawals, and visual (viewshed) impact—under each decarbonization pathway. Each notebook follows a Monte Carlo framework to capture uncertainty in either the conversion factor (land-use, avian mortality), the dispatch-derived activity (water withdrawals), or the spatial placement of projects (viewshed). Each notebook produces a stacked-bar figure with a credible interval and an Excel-formatted summary table. All four are run on the outputs of `Hourly_Installed_Capacity.R` (for capacity-driven impacts) and `dispatch_curve_base_v3.R` (for dispatch-driven impacts).

8.1 Land-use change

Notebook. `Landuse.ipynb`.
Overview. The notebook estimates additional habitat loss (million hectares) from new capacity built between 2025 and 2050 in Solar, Onshore Wind, Canadian Hydro (reservoirs), SMRs, and new natural gas.
Steps.

1. Load all pathway sheets from `Decarbonization_Pathways.xlsx` and stack into one DataFrame.

2. Manually inject "Canadian Hydro" additions for pathway B3 (0 MW in 2025 → 3,692 MW in 2050) since this column is not part of the standard pathway table.
3. Compute capacity change (2050 minus baseline) per technology and pathway.
4. For $n = 1000$ Monte Carlo draws, sample land-use factors (ha/MW) per technology from the following distributions:
   - Canadian Hydro: `Uniform(43.1, 146.7)`.
   - New natural gas: fixed at 0.032 ha/MW.
   - SMR: fixed at 0.017 ha/MW.
   - Onshore Wind: Gamma(α=3.695, θ=9.382).
   - Solar: Gamma(α=4.25, θ=0.82).

   Multiply by the capacity change and convert to million hectares.
5. Compute per-technology means and 5/95% credible intervals on totals.
6. Plot a stacked bar with asymmetric error bars and save `land_use.png`; write the summary to an excel file (`land_use_summary.xlsx`).

Key Code.

Listing 47: Monte Carlo sampling and bar plot

```python
# distribution parameters (ha per MW)
dist_params = {
    'Canadian Hydro': ('uniform', (43.1, 146.7)),
    'New NG':         ('fixed',   0.032),
    'SMR':            ('fixed',   0.017),
    'Onshore Wind':   ('gamma',   (3.695, 9.382)),
    'Solar':          ('gamma',   (4.25, 0.82)),
}

# capacity change 2025 -> 2050
cap_diff = df1[techs] - df0[techs]

# Monte Carlo: n=1000 sims
samples = np.zeros((n_sims, len(pathways), len(techs)))
for i in range(n_sims):
    samp = pd.DataFrame(index=cap_diff.index, columns=techs, dtype=float)
    for tech, (kind, params) in dist_params.items():
        if kind == 'fixed':
            samp[tech] = params
        elif kind == 'uniform':
            samp[tech] = np.random.uniform(*params, size=len(cap_diff))
        elif kind == 'gamma':
            samp[tech] = np.random.gamma(*params, size=len(cap_diff))
    land_diff_ha = cap_diff.multiply(samp, axis=0)
    samples[i, :, :] = (land_diff_ha.loc[pathways, techs] / 1e6).values
```

8.2 Avian mortality

Notebook. `Avian_mortality.ipynb`.
Overview. The notebook estimates millions of bird and bat fatalities attributable to new Onshore Wind and Solar capacity additions between 2025 and 2050.
Steps.

1. Load and stack pathway data; inject B3 Canadian Hydro as above (kept in the calculation only for layout consistency—no avian mortality is attributed to reservoir hydropower).
2. Compute capacity change (2050 minus baseline) for Onshore Wind (which contributes both "bird" and "bat" mortality) and Solar ("bird" mortality only).

3. For $n = 1000$ Monte Carlo draws, sample mortality rates per MW from:
   - Onshore Wind bird rate: Gamma(α=0.20, θ=2.54).
   - Onshore Wind bat rate: Gamma(α=1.538, θ=4.160).
   - Solar bird rate: Lognormal($\mu = \ln(1.214)$, σ=1.409).

   Multiply each rate by the corresponding capacity change.
4. Sum per pathway and convert to millions.
5. Compute means and 5/95% CI of the total, plot stacked-bar with CI, and save `avian_deaths.png`; write per-category and total 10/90% bounds to `avian_deaths_summary.xlsx`.

Key Code.

Listing 48: Sampling bird/bat mortality

```
for s in range(n_sims):
    bird_on = np.random.gamma(0.20, 2.54)
    bat_on  = np.random.gamma(1.538, 4.160)
    s_rate  = np.random.lognormal(np.log(1.214), 1.409)

    samples[s, :, 0] = bird_on * cap_diff["Onshore Wind"].values
    samples[s, :, 1] = bat_on  * cap_diff["Onshore Wind"].values
    samples[s, :, 2] = s_rate  * cap_diff["Solar"].values
samples /= 1e6   # convert to millions
```

8.3 Water withdrawals

Notebook. `Water_withdrawals.ipynb`.

Overview. The notebook computes total water withdrawals (trillion gallons) over 2025–2050 for new SMRs (water-cooled) and new natural-gas generation (dry-cooled). Uncertainty enters through the across-simulation variability in dispatch outputs (`SMR_TWh`, `New_Fossil_Fuel_TWh`) rather than through the per-technology water-use factor, which is held at a fixed midpoint.

Steps.

1. Read the yearly dispatch results (`Yearly_Results.csv`, filtered to 2025–2050).
2. Sum SMR and new-fossil generation per (Simulation, Pathway).
3. Multiply by water-use factors (gal MWh^{-1}):
   - Water-cooled SMRs: 740 gal/MWh.
   - Dry-cooled new natural gas: mean$(15, 50) = 32.5$ gal/MWh.

   Convert to trillion gallons.
4. Compute per-pathway mean and 10/90% credible interval over simulations.
5. Plot stacked-bar with asymmetric CI error bars and save `water_withdrawals.png`; write to an excel file `water_withdrawals_summary.xlsx`.

Key Code.

Listing 49: Water withdrawals calculation

```
import numpy as np
sim_path = (
    yr.groupby(['Simulation','Pathway'])
      [['SMR_TWh', 'New_Fossil_Fuel_TWh']]
      .sum()
)

# water-use factors (gal/MWh midpoints)
```

```
water_mid = {
    'SMR_TWh':              740,
    'New_Fossil_Fuel_TWh': np.mean([15, 50])  # 32.5 gal/MWh, dry-cooled gas
}

# (TWh) * (gal/MWh) -> million gallons; then divide by 1e6 -> trillion gallons
water_mgal = sim_path.mul(pd.Series(water_mid))
water_tr3  = water_mgal / 1e6

mean_tech  = water_tr3.groupby('Pathway').mean()
low_total  = water_tr3.sum(axis=1).groupby('Pathway').quantile(0.10)
high_total = water_tr3.sum(axis=1).groupby('Pathway').quantile(0.90)
```

8.4 Visual impacts (population within viewshed)

Notebook. `Visual_impacts.ipynb`.

Overview. The notebook estimates how many people (in millions) live within project viewshed buffers for new imports infrastructure, offshore wind, and solar. It uses real geographic data and population rasters rather than analytical assumptions, and is therefore the slowest of the four ecological notebooks.

Steps.

1. Compute capacity change (2050 minus baseline) for each of: Imports QC, Imports NYISO, Offshore Wind, Solar.
2. Load New England and source-region polygons (U.S. Census GeoJSON, reprojected to EPSG:5070) and open the ORNL LandScan USA 2021 daytime raster.
3. Define per-technology buffer radii and project sizes:
   - Imports QC, Imports NYISO: 17-mile buffer, 1,200 MW per project.
   - Offshore Wind: 25-mile buffer, 1,000 MW per project.
   - Solar: 3-mile buffer, 100 MW per project.
4. For $n = 10$ Monte Carlo simulations (small because raster masking is expensive) and each pathway-by-technology combination: sprinkle random project points within the appropriate source region (Québec-bordering states, NYISO-bordering states, offshore-NE region, or all of New England for solar), buffer by the tech-specific radius, mask the LandScan raster, and sum the population inside each buffer.
5. Convert to millions of people; compute mean and 5/95% CI per pathway.
6. Plot stacked-bar with CI and save `visual_impacts.png`; write the summary to `viewshed_summary.xlsx`.

Key Code.

Listing 50: Monte Carlo exposure via raster masking

```
buffers_m = {
    "Imports QC":     17 * 1609.34,
    "Imports NYISO": 17 * 1609.34,
    "Offshore Wind": 25 * 1609.34,
    "Solar":           3 * 1609.34,
}
proj_size = {
    "Imports QC":     1200,
    "Imports NYISO": 1200,
    "Offshore Wind": 1000,
    "Solar":           100,
}
n_mc = 10

for s in range(n_mc):
    for i, pw in enumerate(pathways):
        for j, tech in enumerate(techs):
            Δ = cap_diff.at[pw, tech]
```

```
                pop_sum = 0.0
                if Δ > 0:
                    n_proj = int(np.ceil(Δ / proj_size[tech]))
                    # sprinkle random points within the appropriate source region
                    pts = sample_points_in_region(region, n_proj)
                    for p in pts:
                        buf   = p.buffer(buffers_m[tech])
                        geoms = gpd.GeoSeries([buf], crs=states.crs).to_crs(src.crs).geometry
                        arr, _ = rasterio.mask.mask(src, geoms, crop=True, nodata=0)
                        pop_sum += arr.sum()
                mc[s, i, j] = pop_sum
mc /= 1e6   # convert to millions of people
```

8.4.1 Insights

- **Land-use:** Solar and onshore wind drive the largest habitat conversion, with B3 highest overall once reservoir land take is included.
- **Avian mortality:** Onshore wind causes the bulk of bird and bat fatalities; solar contributions are smaller but non-negligible.
- **Water withdrawals:** Water-cooled SMR cooling dominates water use versus dry-cooled new natural gas.
- **Visual impacts:** Imports infrastructure and offshore wind buffers potentially affect millions of residents in New England, depending on project siting.

9 Figure Generation and Post-processing

This section documents the Python (Jupyter) notebooks used to generate the main-text and SI figures from the PHASED model outputs and the cost-table pipeline. Each notebook reads pre-computed CSV outputs (no model re-runs are required) and renders publication-quality figures.

9.1 Generation profiles

Notebook. `Generation.ipynb`.
Inputs.

- `Hourly_Results_sim*_path*_time*.csv`: one representative hourly result file per (simulation, pathway).
- `Yearly_Results.csv`: aggregated yearly summaries.

Outputs.

- `Generation_Pathways.svg`: annual generation stacked-area panels (one per pathway, with demand and curtailment overlays).
- `Sample_week_comp_B1_sim831.svg`: a sample-week hourly dispatch plot showing the stacked generation mix, battery charging/discharging, demand, curtailment, and state-of-charge.
- `Battery_Discharge.svg`: per-pathway annual battery discharge bar charts.
- `legend.svg`: a stand-alone legend used by the multi-panel figures.

Key steps. The notebook builds the stacked area by merging hourly columns for clean technologies, fossil, and imports). The notebook prefers calibrated columns when available, computes hourly battery deltas from `Calibrated_Storage_status`, and overlays the demand and curtailment lines. The colour scheme matches that used by the installed-capacity figure (`installed_capacity_retirement_schedule.ipynb`, Section on Decarbonization Pathways).

9.2 Energy storage

Notebook. `Energy_Storage.ipynb`.
Inputs. `Battery_Data.csv` (written by the dispatch-curve results processing step).
Outputs. `Energy_storage.svg`: a bubble scatter plot with battery discharges (GWh) on the x-axis, mean renewable penetration (%) on the y-axis, and CO_2 emissions (thousand tonnes) encoded as bubble size, one bubble per (Pathway, Year) cell.
Key steps.

Listing 51: Energy storage bubble plot core

```
yearly_results = (
    yearly_results
      .groupby(['Pathway', 'Year'])
      [['RP_mean', 'Battery_Discharges_GWh', 'Shortages_TWh', 'CO2_mtons']]
      .mean()
      .reset_index()
)
yearly_results['RP_mean_percent'] = yearly_results['RP_mean'] * 100

for pathway in unique_pathways:
    subset = yearly_results[yearly_results['Pathway'] == pathway]
    plt.scatter(subset['Battery_Discharges_GWh'],
                subset['RP_mean_percent'],
                s=subset['CO2_mtons'] * 0.01,
                color=pathway_colors[pathway])
```

9.3 County-level air emissions

Notebook. `Emissions.ipynb`.
Inputs.

- `Air_Emissions_County_Level_Existing.csv` and `Air_Emissions_County_Level_New.csv`: per-facility, per-county NPVs of air emission costs from the air-pollutant cost script.
- `New_England_county_boundaries.json` and `US.json`: U.S. Census GeoJSON files for spatial joins and mapping.

Outputs. A multi-panel choropleth showing NPV of air-emission costs by New England county and pathway, with a categorical legend ($0–100 k, 100 k–500 k, 500 k–1 M, 1 M–2 M, 2 M–5 M, 5 M–10 M, >10 M); plus a county-level mortality comparison bar chart for a curated selection of high-impact counties.
Key steps.

Listing 52: County-level emissions summary and choropleth

```
# 1. Sum per (County, State, Pathway, Simulation), then average across simulations
Air_Emissions_County_Level_Summary = (
    Air_Emissions_County_Level
      .groupby(['County', 'State', 'Pathway', 'Simulation'])
      .agg({...})
      .reset_index()
      .groupby(['County', 'State', 'Pathway'])
      .mean()
      .reset_index()
)

# 2. Merge with county geometries and plot one choropleth per pathway
Air_Emissions_County_Level_Summary = (
    Air_Emissions_County_Level_Summary
      .merge(county_boundaries, on=['County', 'State'], how='left')
)
Air_Emissions_County_Level_Summary = gpd.GeoDataFrame(
    Air_Emissions_County_Level_Summary, geometry='geometry'
)
```

9.4 Total costs and B1-difference distributions

Notebook. `Total_Costs.ipynb`.
Inputs. For each of the three discount rates (1.5%, 2%, 2.5%):

- `All_Costs.csv`: across-simulation means by pathway and cost type.
- `All_Costs_per_Simulation.csv`: per-simulation B1-differences, used for ridgeline/violin plots.

Outputs.

- `Total_Costs_Comparison.svg`: stacked-bar with min/max error bars, side-by-side panels for two discount rates.
- `Total_Costs_Diff.svg` and `Total_Costs_Diff_Ridgeline.svg`: ridgeline plots of Δ total cost vs. B1 per pathway, with Gaussian or KDE fits and 5/95% interval markers.

Key steps. Costs are filtered to exclude pathway A (status quo) and (for B1-difference plots) pathway B1 itself; pathway-wise IQR-based outlier removal (1.5 $\times$ IQR) mirrors the R-side aggregation in `13_Total_Costs_and_Uncertainty_Analysis.R`. Pairwise Kolmogorov-Smirnov tests and Wasserstein distances between pathway cost distributions are computed in a separate cell for the manuscript's discussion of distributional differences.

9.5 Correlated uncertainty and global sensitivity

Notebook. `Uncertainty_analysis.ipynb`.
This notebook implements the methodological centrepiece of the manuscript and is documented in the Total Costs section above. In brief, it pools `All_Costs_per_Simulation.csv` across the three discount rates, computes pairwise Spearman correlations with hierarchical clustering and significance markers, and fits a random-forest regressor with permutation importance (and bootstrap CIs) to quantify global sensitivity of total costs to each cost driver, both pooled and per pathway. The corresponding SI figures are:

- `Fig_SI1_Correlation_Spearman_Clustered_LightDiag_NoCbar.svg` (+ `Fig_SI1_Colorbar.svg`).
- `Fig_SI2_Global_Sensitivity_PermFast.svg` (pooled).
- `Fig_SI2_Global_Sensitivity_byPathway.svg` (per pathway).

.